



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

Wydział Informatyki

Projekt dyplomowy

System Obsługi Dziekanatów

Deanary Service System

Autorzy:	Dominik Grzesik, Marcin Mikuła, Sebastian Kozak, Szymon Słota
Kierunek studiów:	Informatyka
Opiekun pracy:	dr inż. Jacek Dajda, prof. AGH

Kraków, 2025

Spis Treści

1	Cel prac i wizja produktu	6
1.1	Przegląd dostępnych rozwiązań	7
1.1.1	USOS	7
1.1.2	Poczta AGH	9
1.1.3	Witryna internetowa AGH - agh.edu.pl	10
1.1.4	Nasze rozwiązanie	12
1.2	Wskazania współpracujących zewnętrznych systemów	13
2	Zakres funkcjonalności	14
2.1	Użytkownicy i osoby	14
2.1.1	Student	14
2.1.2	Starosta	15
2.1.3	Przewodniczący Wydziałowej Rady Samorządu Studentów	15
2.1.4	Pracownik dziekanatu	16
2.1.5	Admin	16
2.2	Wymagania funkcjonalne	18
2.2.1	Aplikacja studencka (mobilna)	18
2.2.2	Aplikacja dziekanatu (webowa)	25
2.3	Opis funkcji użytkowników	35
2.3.1	Rejestracja - aplikacja webowa	35
2.3.2	Rejestracja - aplikacja mobilna	35
2.3.3	Logowanie	36
2.3.4	Wyświetlanie wpisów	36
2.3.5	Tworzenie wpisów	36
2.3.6	Edytowanie wpisów	37
2.3.7	Filtrowanie zamieszczonych wpisów	37
2.3.8	Zmiana języka wyświetlania w aplikacji webowej	37
2.3.9	Zmiana języka wyświetlania w aplikacji mobilnej	38
2.3.10	Usuwanie wpisów w aplikacji webowej	38
2.3.11	Nadawanie ról starostom	38
2.3.12	Usuwanie uprawnień starosty	39
2.3.13	Nadawanie uprawnień p. WRSS	39
2.3.14	Usuwanie uprawnień p. WRSS	39
2.3.15	Usuwanie kont użytkowników	40
2.3.16	Zatwierdzanie konta pracownika	40
2.3.17	Dodawanie wydziałów	41
2.3.18	Dodawanie kierunków	41
2.3.19	Usuwanie wydziałów	42

2.3.20	Usuwanie kierunków	42
2.3.21	Edytowanie wydziałów	42
2.3.22	Edytowanie kierunków	43
2.3.23	Edycja profilu	43
3	Wybrane aspekty realizacji	44
3.1	Ogólny opis projektu	44
3.2	Stos technologiczny	45
3.2.1	Backend	45
3.2.2	Rozdzielenie aplikacji mobilnej i webowej	46
3.2.3	Aplikacja webowa	46
3.2.4	Aplikacja mobilna	47
3.2.5	Podsumowanie	47
3.3	Struktura systemu - Backend	48
3.3.1	Udostępniane usługi	50
3.3.2	Model danych	52
3.4	Struktura systemu - aplikacja webowa	54
3.4.1	Konfiguracja aplikacji	55
3.4.2	Kod źródłowy	56
3.5	Struktura systemu - aplikacja mobilna	61
3.5.1	Konfiguracja aplikacji	61
3.5.2	Kod źródłowy	63
3.6	Bezpieczeństwo	66
3.7	Budowanie i wdrażanie	68
3.8	Testowanie funkcjonalności oraz jakości kodu	69
4	Organizacja pracy	70
4.1	Charakterystyka projektu i sposób jego realizacji	70
4.2	Osoby i ich role w projekcie	71
4.3	Zespół i podział obowiązków	71
4.3.1	Forma pracy zespołowej	71
4.3.2	Sebastian Kozak	71
4.3.3	Marcin Mikuła	71
4.3.4	Szymon Słota	72
4.3.5	Dominik Grzesik	72
4.3.6	Zespół	72
4.4	Wykorzystane narzędzia do organizacji pracy	73
4.4.1	Git	73
4.4.2	Bitbucket, Github	74
4.4.3	Jira	75
4.4.4	Teams	75
4.4.5	Discord	75
4.4.6	Messenger	75
4.4.7	Email	75
4.4.8	Google Drive	75
4.4.9	Overleaf, diagrams.net, Balsamic, Dokumenty Google	75
4.5	Przebieg prac	76
4.6	Występujące ryzyka	79

5	Wyniki projektu	80
5.1	Podsumowanie	80
5.2	Prezentacja wyników projektu	81
5.2.1	Aplikacja webowa	82
5.2.2	Aplikacja mobilna	88
5.2.3	Porównanie wyglądu aplikacji na systemach iOS i Android	95
5.3	Podsumowanie wraz z własną subiektywną oceną	96
5.4	Recenzja i opinia klienta	97

Rozdział 1

Cel prac i wizja produktu

Jednym z najczęściej doświadczanych problemów studentów jest przeładowana wiadomościami skrzynka pocztowa. Każdy student od momentu rozpoczęcia studiów zakłada konta na wielu serwisach potrzebnych do realizacji kursów. Wadą tego rozwiązania są napływające wiadomości z tychże serwisów, choć kolejne semestry mijają, realizowane są kolejne kursy, konta na nieużywanych portalach pozostają i nieustannie na skrzynce pocztowej pojawiają się na samej górze powiadomień. Istnieją możliwości ograniczania takiego spamu, można usuwać konta, blokować wiadomości, jednak wymaga to dodatkowego zaangażowania i czasu. W rezultacie, studenci dziennie otrzymują dziesiątki maili będących spamem, przysyłających istotne wiadomości z dziekanatu, od prowadzących kursy czy innych studentów. Informacje z uczelni przekazywane w wielu miejscach - Poczta AGH, USOS bądź strona uczelni, wszystkie z wymienionych wykorzystywane są również do innych celów. Niektóre z tematów wymagają osobistego zgłoszenia się studenta w dziekanacie, co powoduje narastające niezadowolenie studentów oraz pracowników.

W celu ułatwienia dostępu i udoskonalenia przepływu informacji na uczelni należałoby zebrać takie posty i aktualności w jedno miejsce. Dodatkowo w celu zwiększenia przejrzystości, informacje pogrupowane byłyby wedle roku bądź kierunku. Wtedy student byłby w stanie widzieć tylko te posty, które go dotyczą bądź interesują.

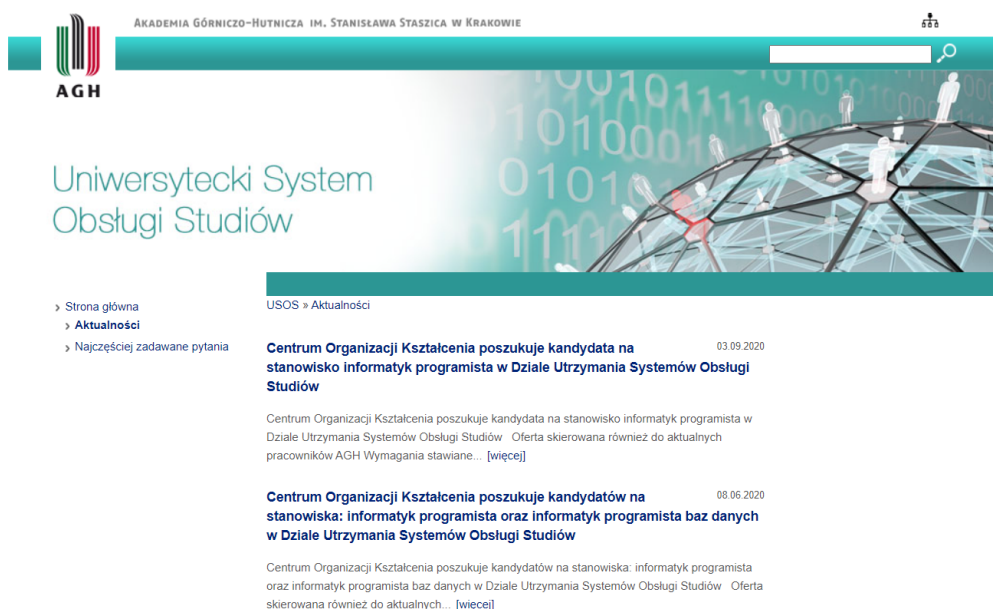
Motywacją oraz celem projektu jest usprawnienie komunikacji między studentami a dziekanatami Akademii Górniczo-Hutniczej w Krakowie, stąd powstała potrzeba stworzenia portalu w formie forum, który zostanie nazwany - Studencki Panel Informacyjny.

1.1. Przegląd dostępnych rozwiązań

Istnieją już gotowe narzędzia, niestety w większości nie spełniają wszystkich wymagań. Daleko im do ideału produktu, który w pełni zadowoliliby wymagającego użytkownika. Studenci podczas poszukiwań ogłoszeń lub informacji od swoich dziekanatów szukają informacji na wielu platformach uczelni.

1.1.1. USOS

Pierwszą, najbardziej oczywistą opcją wydaje się system USOS - następca wirtualnego dziekanatu. Większość studentów właśnie tutaj rozpocznie poszukiwania informacji. Student znajdzie wszystkie ogłoszenia w jednym miejscu nawet te ogłoszenia, które kompletnie go nie dotyczą.



1.1: USOS dostępny dla każdego użytkownika

Przechodząc na podstronę "Aktualności" student otrzymuje listę wiadomości dotyczących całej uczelni. Kliknięcie na komunikat przenosi go na oddzielny widok, gdzie znajdują się szczegółowe informacje o ogłoszeniu.

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie

AKTUALNOŚCI KATALOG MÓJ USOSWEB DLA STUDENTÓW DLA PRACOWNIKÓW DLA WSZYSTKICH

DOKUMENTY
→ witamy

ostatnia migracja danych: ok. 5 godzin temu

Witamy w serwisie USOSweb

Akademii Górniczo-Hutniczej im. Stanisława Staszica w Krakowie

USOSweb to internetowy moduł USOSa przeznaczony dla studentów oraz nauczycieli akademickich.

Dane pomiędzy modułami USOSa (USOSweb a USOSadm dla dziekanatów) aktualizowane są dwa razy na dobę, o godzinie 6:30 oraz 16:30. Dane wprowadzone przez pracowników dziekanatu będą widoczne dla studentów i nauczycieli akademickich po godzinach migracji, natomiast dane wprowadzone przez nauczycieli dla studentów są widoczne od razu.

Do zalogowania się w USOS AGH **wymagane** jest posiadanie konta pocztowego w domenie uczelni. Jeśli nie posiadasz jeszcze swojej uczelnianej skrzynki email, przejdź do strony Gdzie założyć konto e-mail AGH?. W razie problemów z utworzeniem konta AGH lub zresetowaniem do niego hasła, skontaktuj się z pomoc@cri.agh.edu.pl.

Do kontaktów we wszystkich sprawach studenckich należy używać uczelnianego konta pocztowego w domenie student.agh.edu.pl – jest to niezbędne do uwierzytelnienia danych studenta. Dodatkowo maile prosimy podpisywać imieniem, nazwiskiem oraz numerem albumu.

Uwaga! Adresy mailowe:

- [\[numer_albumu@student.agh.edu.pl\]](mailto:[numer_albumu@student.agh.edu.pl])
- [\[wybrany przez studenta login@student.agh.edu.pl\]](mailto:[wybrany_przez_studenta_login@student.agh.edu.pl])

są równoznaczne i poprawne.

Ważne dane kontaktowe

Rodzaj sprawy/problemu:	Dane kontaktowe:
USOSweb (problemy techniczne)	pomoc-usos@agh.edu.pl
Przypisane przedmioty	...

1.2: USOS dostępny dla zalogowanego studenta

Po przejściu procesu logowania, w aktualnościach widnieje jedynie informacja, że trzeba posiadać konto, żeby się zalogować, oraz tabela z danymi kontaktowymi. Natomiast po lewej stronie znajduje się panel z niepowiązаныmi ze sobą, niejasnymi możliwościami przekierowania.

Zalety:

- Oficjalny system używany na AGH
- Dostęp do wszystkich ogłoszeń
- Student jest w stanie szybko znaleźć informacje

Wady:

- Zawierający nieaktualne ogłoszenia
- Wszystkie wiadomości w jednym miejscu
- Brak możliwości jakiegokolwiek filtrowania informacji

1.1.2. Poczta AGH

Drugą możliwością, po którą sięgają studenci jest poczta AGH. Strona poczty to nie tylko lista maili przychodzących od dziekanatów, prowadzących zajęcia i starostów, ale również od wszystkich narzędzi, których student używał wykorzystując maila uczelnianego, chociażby w przypadku, gdy potrzebował licencji, którą uczelnia udostępnia.

Wiadomości od 1 do 50 z 1171	
Adresat	Czw 12:59
• Accepted: Konsultacje	
Adresat	Czw 12:59
• Konsultacje	
Balsamiq Cloud	Śr 13:03
• XYZ has invited you to join the Balsamiq Cloud Sp...	
Atlassian	Wt 10:33
• Your subscription has been changed	
Atlassian	Wt 10:33
• Your subscription has been changed	
Miro	Wt 09:38
• Kolunie team Weekly Digest	
Adresat	Pn 10:53
• Konsultacje	
Adresat	Pn 10:51
• Accepted: Konsultacje	
Adresat	Pn 10:51
• Konsultacje	
XYZ (z UPeL)	2023-04-14 15:00
• UXWeb: Przygotowanie do Laboratorium 4.	
Clickup	2023-04-14 11:55
• Your subscription has been changed	

1.3: Skrzynka pocztowa AGH

Zalety:

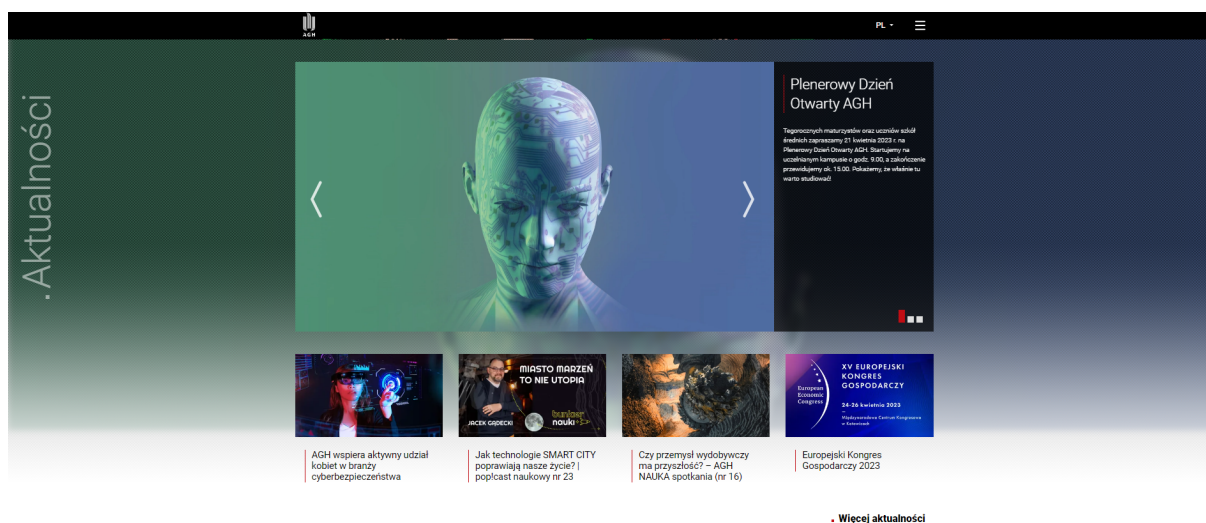
- Łatwy i szybki dostęp do informacji
- Emaila dotyczące licencji uczelnianych zebrane w jednym miejscu

Wady:

- Brak skalowania dla małych rozdzielczości ekranu
- Brak wsparcia dla urządzeń mobilnych - aplikacja stricte dla użytkowników komputerów i laptopów
- Cecha skrzynki pocztowej - spam miesza się z istotnymi ogłoszeniami i wiadomościami

1.1.3. Witryna internetowa AGH - agh.edu.pl

W celu przeglądnięcia ogólnych informacji o aktualnościach uczelni, student, pracownik bądź osoba zainteresowana z zewnątrz mogą utworzyć witrynę agh.edu.pl. Strona uczelni udostępnia ogólne informacje i ogłoszenia dotyczące wszystkich kierunków oraz nowych kandydatów na studentów, chociażby post o dniach otwartych. Choć strona jest często aktualizowana, zawiera jedynie wydarzenia ogólnie dotyczące całego AGH, nie ma możliwości szukania informacji w obrębie wydziałów.



1.4: Strona główna agh.edu.pl

Aktualności

Strona główna / Aktualności

energetyka (3) granty (11) nagrody (16) projekty studenckie (20) smog (2) sport (7)
 środowisko (3) superkomputery (3) sztuczna inteligencja (3) zrównoważony rozwój (1)

Szukana fraza

Data od
dd.mm.rrrr

Data do
dd.mm.rrrr

Szukaj

1.5: Strona “Więcej aktualności”

AGH wspiera aktywny udział kobiet w branży cyberbezpieczeństwa

W AGH odbyło się spotkanie inauguracyjne inicjatywę Women4Cyber – polskiego oddziału wiodącej europejskiej fundacji [...]

21.04.2023



AGH wśród zwycięzców konkursu GOSPOSTRATEG

W ramach konkursu Strategicznego programu badań naukowych i prac rozwojowych „Społeczny i gospodarczy rozwój [...]

19.04.2023



1.6: Strona “Więcej aktualności” - cd

Zalety:

- Informacje ogólne, dotyczące całej uczelni
- Zawartość strony aktualizowana jest na bieżąco
- Nowoczesny design
- Poprawne skalowanie na urządzenia desktopowe i mobilne

Wady:

- Brak możliwości przeglądania ogłoszeń dotyczących wybranego wydziału lub roku

1.1.4. Nasze rozwiązanie

Nasz produkt zbierze wszystkie komunikaty w jedno miejsce - będzie to aplikacja dedykowana wyświetlaniu ogłoszeń dziekanatów. Informacje będą przesyłane do poszczególnych grup, na przykład na dany wydział lub rok. Użytkownik będzie mógł filtrować wiadomości, co skutecznie przyspieszy proces szukania informacji.

Celem prac jest zbudowanie w pełni działających i responsywnych aplikacji usprawniających przepływ informacji na uczelni. Powstałyby dwie aplikacje: mobilna dla studentów pozwalająca na podgląd wiadomości oraz webowa dla pracowników dziekanatów, z której przekazywane byłyby informacje.

W aplikacji wyróżnieni będą użytkownicy tacy jak:

- Student
- Pracownik dziekanatu

Punktem wyjścia dla nas jest student oraz pracownik dziekanatu, jednakże wyróżniamy dodatkowe, mające specjalne funkcje i odpowiednie uprawnienia, role, takie jak:

- Starosta
- Przewodniczący samorządu wydziałowego
- Admin systemu

Wymienieni aktorzy zostali szczegółowo opisani w rozdziale 2.1.

1.2. Wskazania współpracujących zewnętrznych systemów

Prototyp aplikacji działać będzie bez żadnych dodatkowych systemów zewnętrznych. W przypadku chęci wdrożenia aplikacji do systemu uczelnianego, konieczne będzie wykorzystanie API USOS'a oraz Sylabusa AGH, pozwalające na operowanie na rzeczywistych danych użytkowników oraz wydziałów i kierunków.

Takie informacje wykorzystywane byłyby do:

- uwierzytelniania studentów - student nieaktywny nie ma dostępu do aplikacji, osoba z zewnątrz nie może założyć konta
- pobieranie informacji o kierunkach i wydziałach na AGH bezpośrednio z Sylabusa

Rozdział 2

Zakres funkcjonalności

2.1. Użytkownicy i osoby

Projektując naszą aplikację zdecydowaliśmy się wykorzystać technikę person. Persona oznacza wyidealizowany profil użytkownika, do którego kierujemy nasz produkt.

Aktorzy, którzy są częścią naszej aplikacji to:

- Student
- Starosta
- Przewodniczący Wydziałowej Rady Samorządu Studentów
- Pracownik dziekanatu
- Admin

2.1.1. Student

Gość po zalogowaniu przy użyciu maila w domenie @student.agh.edu.pl. Może przeglądać posty, które są mu udostępniane.

Przykładowa persona

- Imię i Nazwisko: Sebastian Kozak
- Wiek: 20
- Identyfikacja płciowa: Inne
- Wykształcenie: Średnie
- Doświadczenie w korzystaniu z podobnych systemów: Brak
- Cel korzystania z systemu:
Chęć bycia na bieżąco z aktualnościami, które mogą go dotyczyć.
- Oczekiwania wobec systemu:
Schludna, łatwa w użytkowaniu aplikacja.

2.1.2. Starosta

Student z uprawnieniami do dodawania, edytowania i usuwania postów w przypisanej mu grupie użytkowników.

Przykładowa persona

- Imię i Nazwisko: Michał Starosta
- Wiek: 21
- Identyfikacja płciowa: Mężczyzna
- Wykształcenie: Średnie
- Doświadczenie w korzystaniu z podobnych systemów: Brak
- Cel korzystania z systemu:
Chęć bycia na bieżąco z aktualnościami, które mogą go dotyczyć oraz publikowanie postów z informacjami dla studentów jego roku i kierunku.
- Oczekiwania wobec systemu:
Łatwa w użytkowaniu aplikacja, pozwalająca na szybkie przesyłanie komunikatów.

2.1.3. Przewodniczący Wydziałowej Rady Samorządu Studentów

Student z uprawnieniami do dodawania, edytowania i usuwania postów w przypisanym mu wydziale.

Przykładowa persona

- Imię i Nazwisko: Adam Nowak
- Wiek: 23
- Identyfikacja płciowa: Mężczyzna
- Wykształcenie: Średnie
- Doświadczenie w korzystaniu z podobnych systemów: Aplikacje typu Instagram
- Cel korzystania z systemu:
Zamieszczanie informacji o wydarzeniach na danych wydziale, nadawanie uprawnień starostom oraz również dostęp do dotyczących go ogłoszeń.
- Oczekiwania wobec systemu:
Aplikacja, która zawierać będzie wszystkie konieczne funkcjonalności.

Student, starosta oraz przewodniczący Wydziałowej Rady Samorządu Studentów to typowe osoby studiujące, w wieku około 22 lat, na co dzień korzystające z platform społecznościowych.

2.1.4. Pracownik dziekanatu

Gość po zalogowaniu przy użyciu maila w domenie @agh.edu.pl. Może dodawać, edytować i przeglądać posty.

Przykładowa persona

- Imię i Nazwisko: Anna Kowalczyk
- Wiek: 50
- Identyfikacja płciowa: Kobieta
- Wykształcenie: Wyższe
- Doświadczenie w korzystaniu z podobnych systemów: Onet, Facebook
- Cel korzystania z systemu:
Zamieszczanie komunikatów dla studentów.
- Oczekiwania wobec systemu:
Nieskomplikowana, prosta w użytkowaniu aplikacja.

Typowy pracownik to osoba około 50 lat, na co dzień nie korzystająca z portali społecznościowych.

2.1.5. Admin

Pracownik będący zarządcą systemu, mający możliwość usuwania lub blokowania kont użytkowników, zatwierdzania kont pracowników, nadawania statusu przewodniczącego WRSS, zarządzaniem listami wydziałów i kierunków.

Przykładowa persona

- Imię i Nazwisko: Marcin Mikuła
- Wiek: 31
- Identyfikacja płciowa: Mężczyzna
- Wykształcenie: Wyższe
- Doświadczenie w korzystaniu z podobnych systemów:
Doświadczenie w moderacji podobnymi systemami.
- Cel korzystania z systemu:
Moderacja kontrola w systemie, dbanie o bezpieczeństwo w aplikacji.
- Oczekiwania wobec systemu:
Aplikacja, której funkcjonalności są wyczerpujące i pozwalają na sprawne zarządzanie systemem.

Admin to osoba starsza od typowego studenta, młodsza jednak od typowego pracownika dziekanatu, posiadająca doświadczenie w zarządzaniu i moderowaniu platformami społecznościowymi.

Użytkownicy naszego systemu to osoby w praktycznie każdym wieku, posiadające lub nieposiadające doświadczenia w korzystaniu z aplikacji społecznościowych. Persony pomogły nam lepiej zrozumieć jakie są potrzeby i oczekiwania naszych użytkowników - aplikacja powinna być przejrzysta, nieskomplikowana w użyciu, intuicyjna oraz łatwa w zarządzaniu.

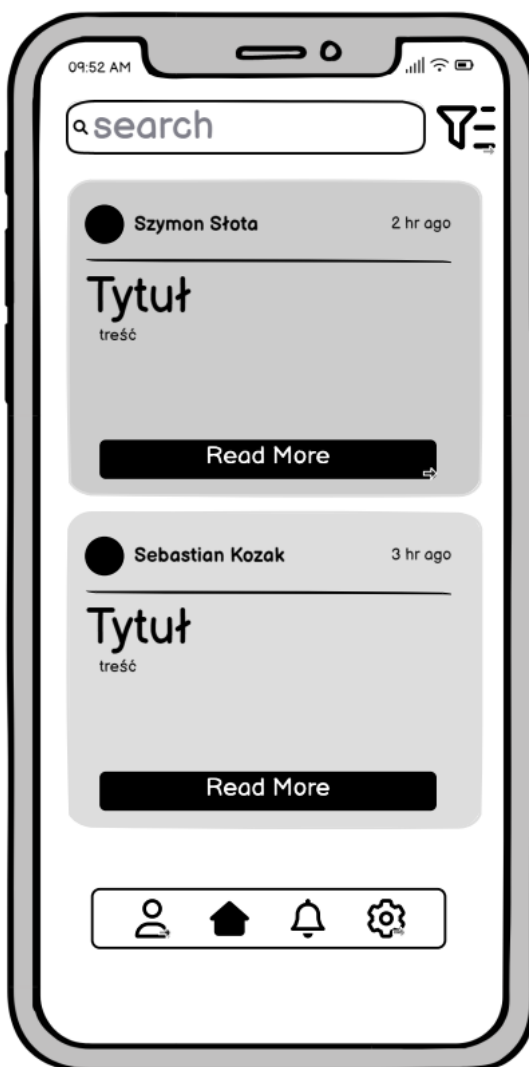
2.2. Wymagania funkcjonalne

Poniżej przedstawiono zebrane w ramach projektu dążenia, uporządkowane według typu aplikacji. Niektóre z nich są zilustrowane makietą, która powstała, na samym początku wytwarzania oprogramowania.

2.2.1. Aplikacja studencka (mobilna)

Wpis publikowany dla danej grupy musi zawierać informacje:

- Kto opublikował wpisy (dziekanat, przewodniczący WRSS, starosta)
- Temat ogłoszenia, tzn. krótki opis, sugerujący zawartość treści postu
- Treść postu, rozwinięcie tematu



2.1: Makieta przedstawiająca widok główny zalogowanego studenta w aplikacji mobilnej, widoczne są 2 przykładowe posty. Każdy z wpisów zawiera nagłówek w postaci autora oraz daty opublikowania go, tytuł, treść i przycisk, który pozwala przejść do widoku szczegółowego

Student:

- Tworzenie własnego konta w aplikacji w domenę AGH, poprzez podanie prawidłowych danych:
 - mail
 - imię
 - nazwisko
 - kierunek wraz z wydziałem oraz rokiem
 - hasło
 - powtórzone hasło

09:52 AM

Sign up

Email

Firstname

Lastname

Faculty, Major

Password

Repeat password

Sign up

Sign in

2.2: Makieta przedstawiająca przykładową rejestrację studenta, po pomyślnym zarejestrowaniu się przeniesie nas do widoku logowania (makieta poniżej)

- Możliwość zalogowania się do aplikacji jest dostępna jedynie przy użyciu poprawnie utworzonego konta w domenie AGH, wymagającego podania prawidłowego hasła



2.3: Makieta przedstawiająca logowanie w aplikacji mobilnej

- Możliwości przypisania się do więcej niż jednego kierunku studiów
- Przeglądanie własnego profilu



2.4: Makieta przedstawiająca profil studenta

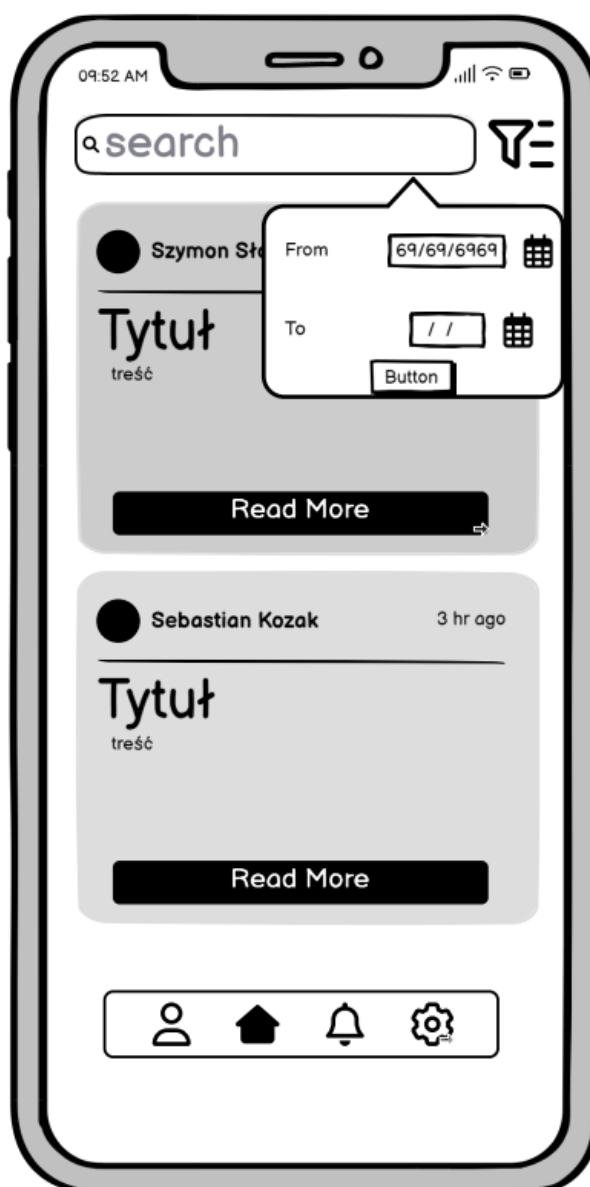
- Edycja własnego profilu
 - modyfikacja roku
 - modyfikacja wydziału
 - modyfikacja kierunku

- Wyświetlanie listy aktualności przypisanych tylko grupie użytkownika



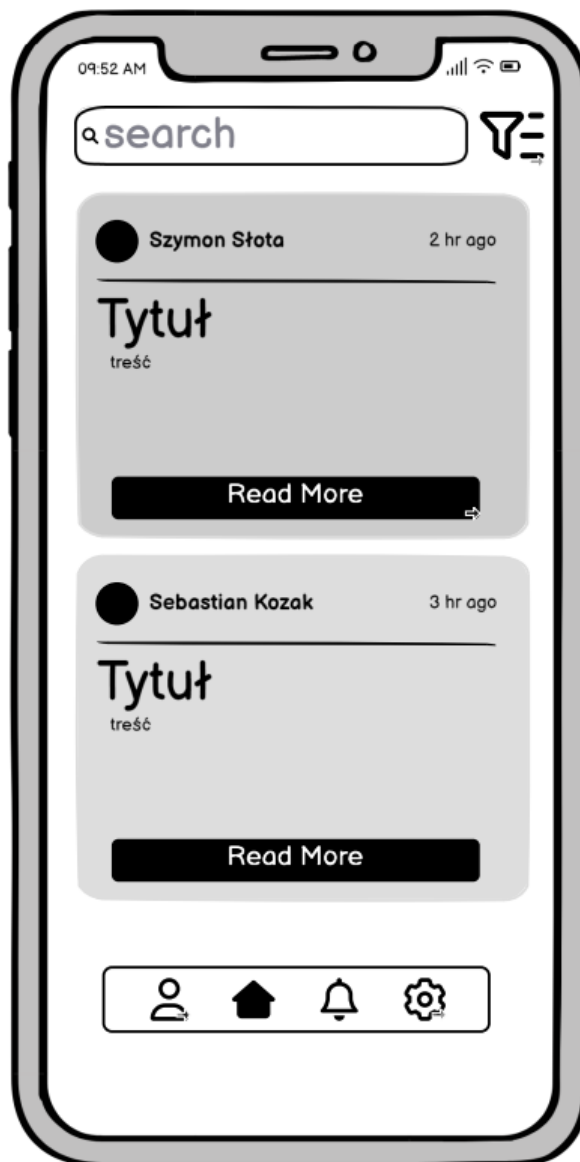
2.5: Makieta przedstawiająca dedykowane posty studenta

- Filtrowanie dodanych ogłoszeń



2.6: Makieta przedstawiająca filtrowanie postów studenta

- Poruszanie się po aplikacji mobilnej za pomocą paska nawigacji



2.7: Makieta przedstawiająca widok główny aplikacji mobilnej, na dole widoku znajduje się pasek nawigacji, który odpowiednio umożliwia nam przejście do: profilu, strony głównej, powiadomień, ustawień

Przewodniczący WRSS oraz Starosta mają wszystkie uprawnienia Studenta.

Wymagania niefunkcjonalne

- Aplikacja dostępna wyłącznie dla zalogowanych użytkowników
- Wymagana rejestracja konta mailem z domeny AGH (osobna ścieżka logowania po domenie @student.agh.edu.pl osobna po domenie pracowniczej @agh.edu.pl)
- Poprawne działanie na systemach IOS i Android
- Skalowanie aplikacji na różnych rozdzielczościach ekranu
- Obsługa grupy użytkowników - studenci AGH (Obsługa dużej liczby połączeń($10^3 +$) oraz odczytów z bazy danych)
- Wyświetlanie aplikacji w dwóch językach: polskim i angielskim

2.2.2. Aplikacja dziekanatu(webowa)

Wymagania funkcjonalne

Rejestracja pracownika w aplikacji:

The screenshot shows a web browser window with the URL <https://red.agh.edu.pl>. The page displays a registration form titled "Zarejestruj się" with a close button (X). The form contains the following fields: "Imię", "Nazwisko", "Email", "Hasło", and "Powtórz hasło". Below these fields is a "Dalej" button. At the bottom of the form, there is a link: "Masz już konto? Zaloguj się!". The background of the page is blurred, showing a login form with a "Zaloguj się" button and the text "System Obsługi Dziekanatów".

2.8: Makieta przedstawiająca rejestrację pracownika, wymagane jest podanie: imienia, nazwiska, maila, hasła dwukrotnie

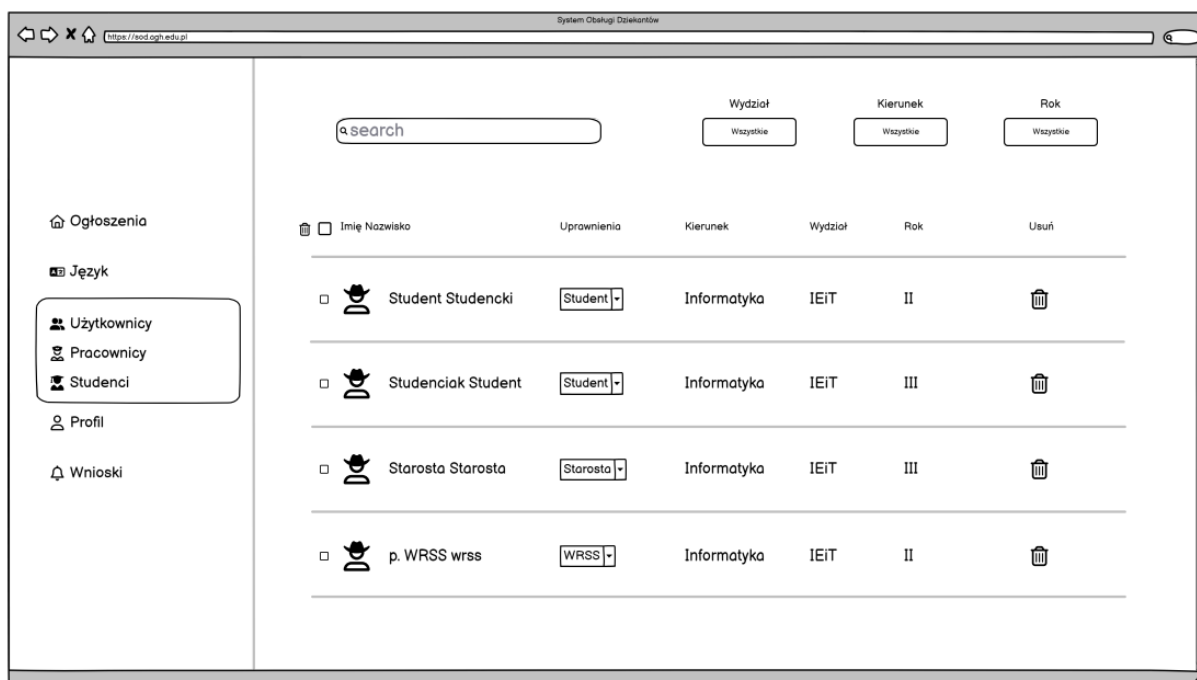
1. Admin dostaje informacje o próbie rejestracji pracownika i może zatwierdzić lub odrzucić wniosek
2. Automatyczne przesłanie maila na podany przez pracownika adres z linkiem aktywacyjnym



2.9: Makieta przedstawiająca powiadomienie o wysłaniu maila aktywacyjnego

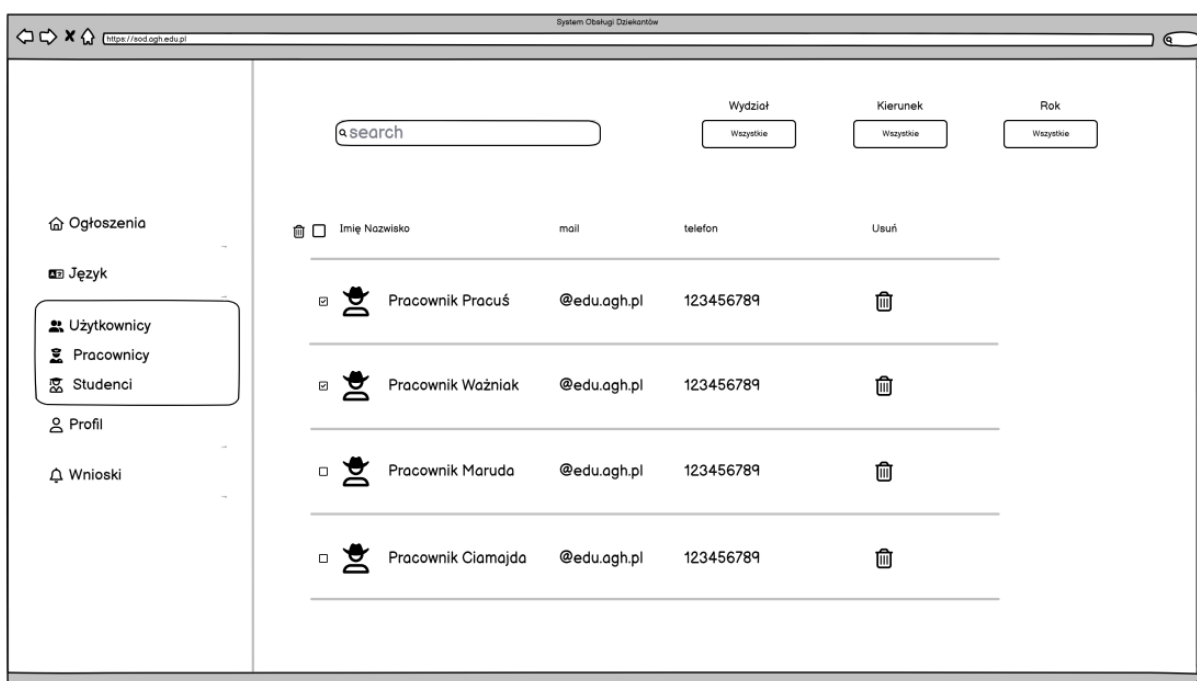
Admin:

- Zatwierdzanie wniosku o utworzenia konta pracowniczego
- Nadawanie roli przewodniczącemu WRSS



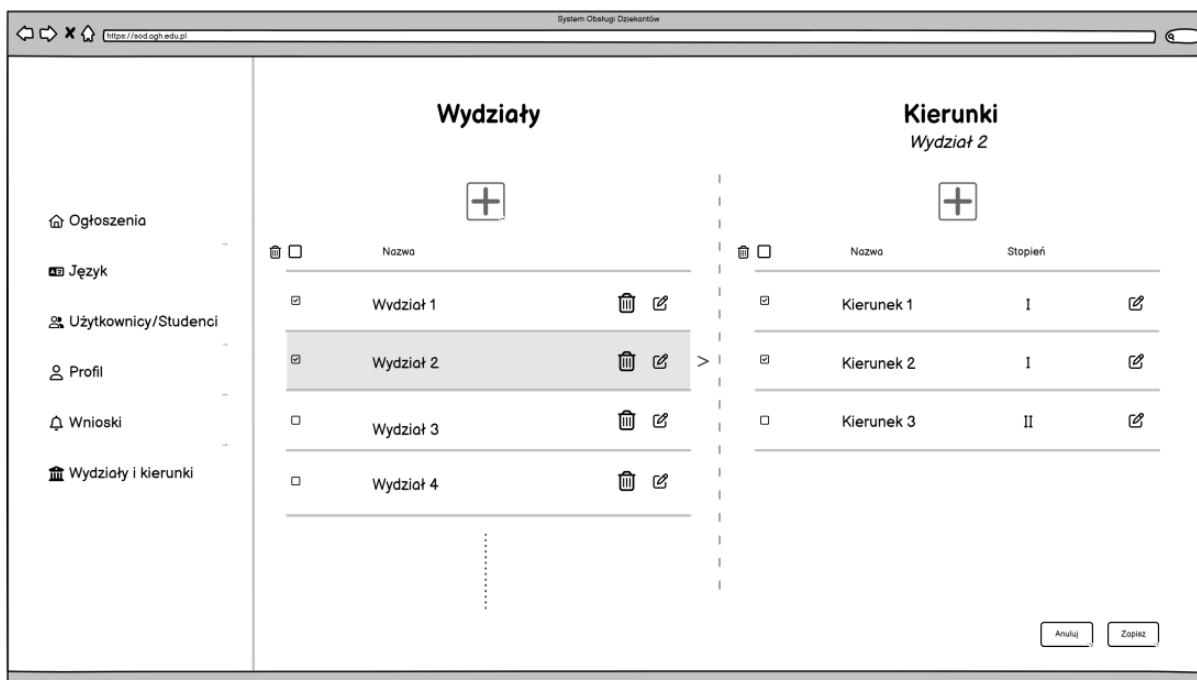
2.10: Makieta przedstawiająca tabelę studentów, na której w każdym wierszu zawarty jest student (imię oraz nazwisko), jego aktualna rola z możliwością zmiany jej za pomocą rozsunięcia menu i wyboru nowej roli, kierunek, wydział, rok oraz przycisk do usunięcia

- Usuwaniam użytkowników jeżeli będzie to konieczne



2.11: Makieta przedstawiająca listę pracowników z możliwością usunięcia ich (ikony kosza)

- Blokowaniem użytkowników jeżeli będzie to konieczne
- Usuwaniem treści w każdej z dostępnych grup jeżeli będzie to konieczne lub komunikat naruszałby zasady funkcjonowania w przestrzeni AGH
- Zarządzaniem listami wydziałów i kierunków



2.12: Makieta przedstawiająca listę wydziałów oraz kierunków

Starosta:

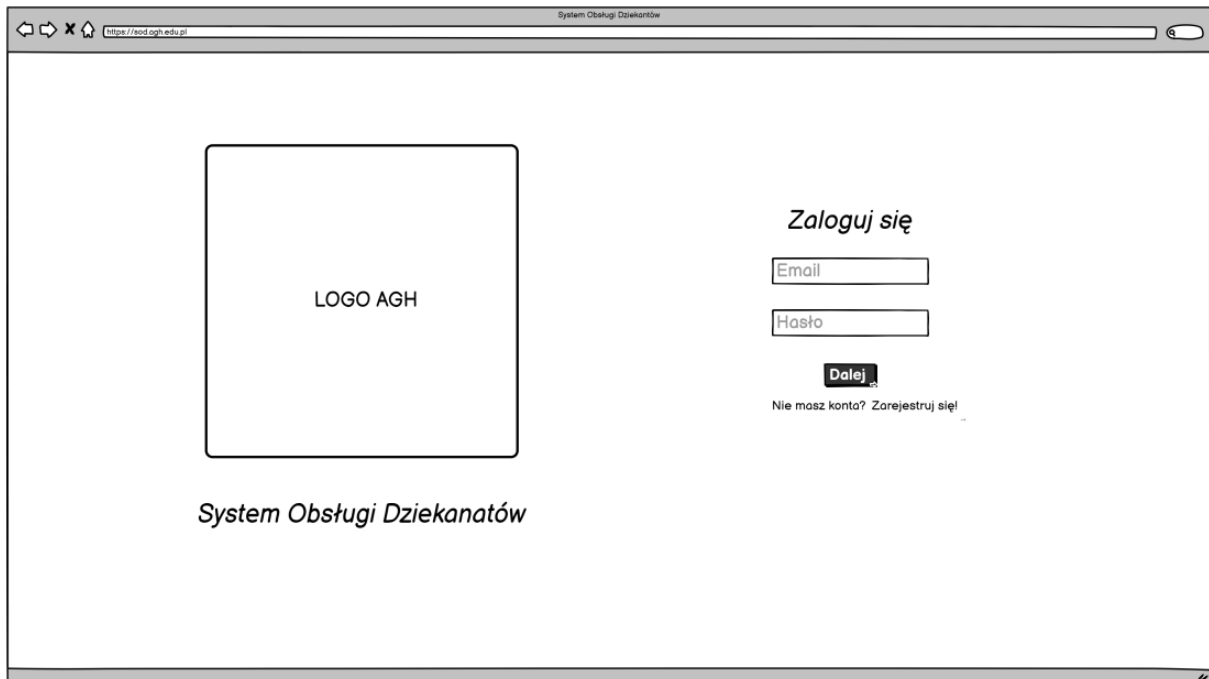
- Tworzenie własnego konta w aplikacji w domenie AGH
- Możliwość zalogowania się do aplikacji wyłącznie za pomocą konta założonego w domenie AGH
- Przeglądanie i edycja własnego profilu
- Wyświetlanie listy aktualności przypisanych tylko grupie użytkownika
- Filtrowanie dodanych ogłoszeń
- Możliwość wstawienia ogłoszenia dla użytkowników należących do jego grupy
- Możliwość edycji dodanych przez niego postów

Przewodniczący rady samorządu:

- Tworzenie własnego konta w aplikacji w domenie AGH
- Możliwość zalogowania się do aplikacji wyłącznie za pomocą konta założonego w domenie AGH
- Nadawanie roli starosty
- Określanie grupy docelowej starosty
- Publikowanie komunikatów do wszystkich grup
- przypisanych do danego wydziału
- Usuwanie uprawnień starosty
- Przeglądanie i edycja własnego profilu
- Wyświetlanie listy aktualności przypisanych tylko grupie użytkownika
- Filtrowanie dodanych ogłoszeń
- Możliwość edycji dodanych przez niego postów

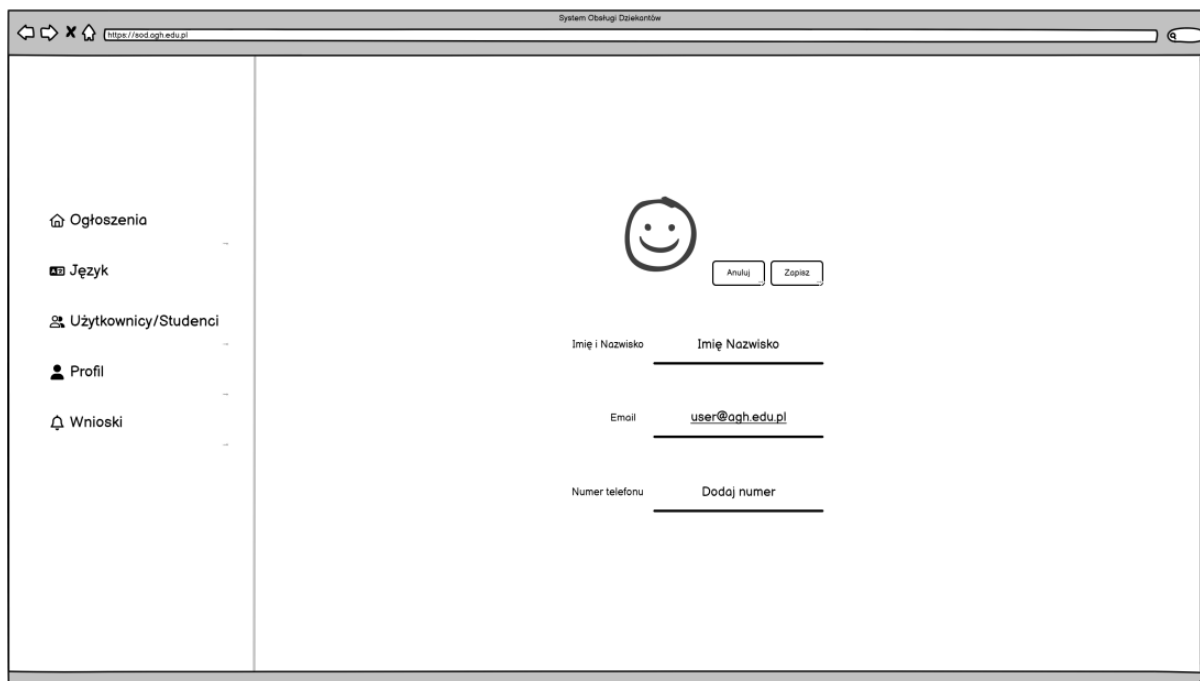
Pracownik dziekanatu:

- Tworzenie własnego konta w aplikacji w domenie AGH
- Możliwość zalogowania się do aplikacji wyłącznie za pomocą konta założonego w domenie AGH



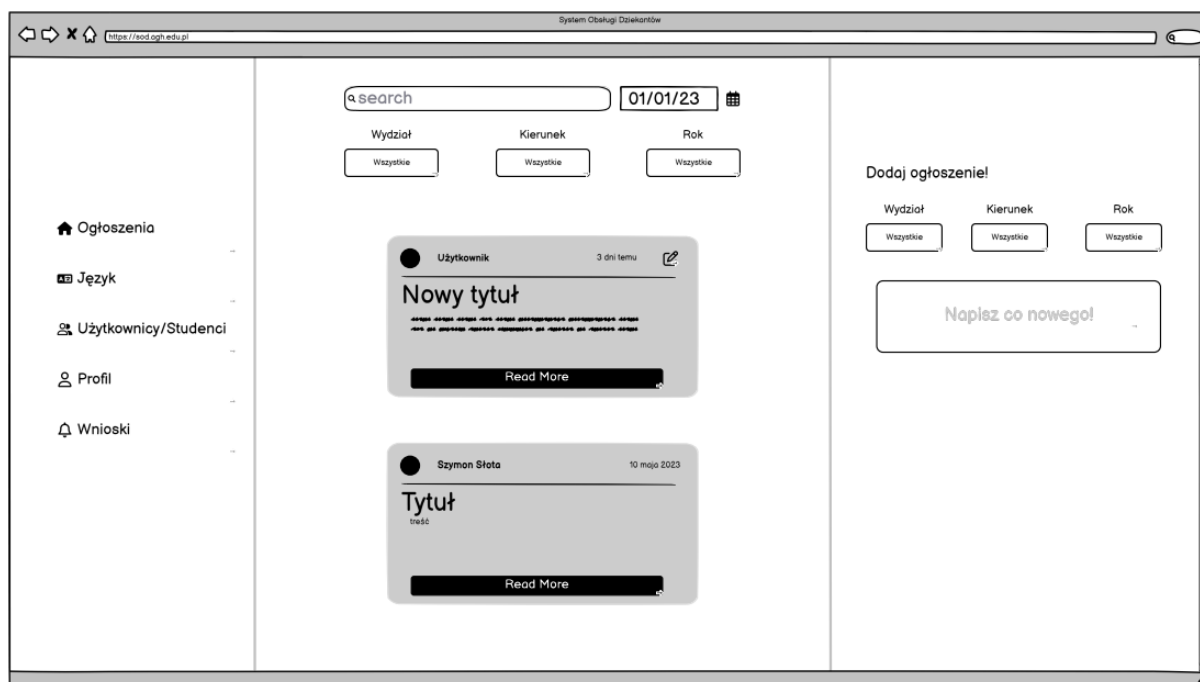
2.13: Makieta przedstawiająca stronę tytułową, po lewej stronie widnieje logo AGH wraz z nazwą aplikacji, natomiast po prawej znajduje się sekcja logowania, w której trzeba wypełnić pola mail oraz hasło aby pomyślnie zalogować się

- Przeglądanie własnego profilu
- Edycja danych osobowych profilu



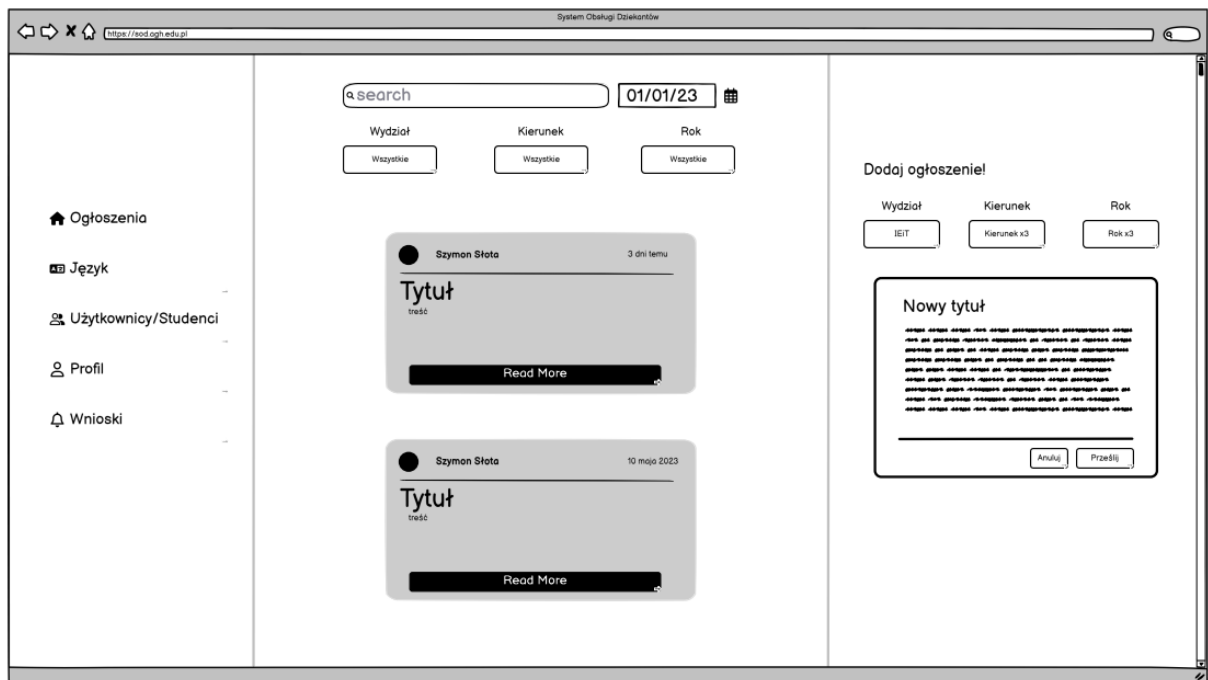
2.14: Makieta przedstawiająca profil wraz z możliwością edycji pracownika

- Przeglądanie postów



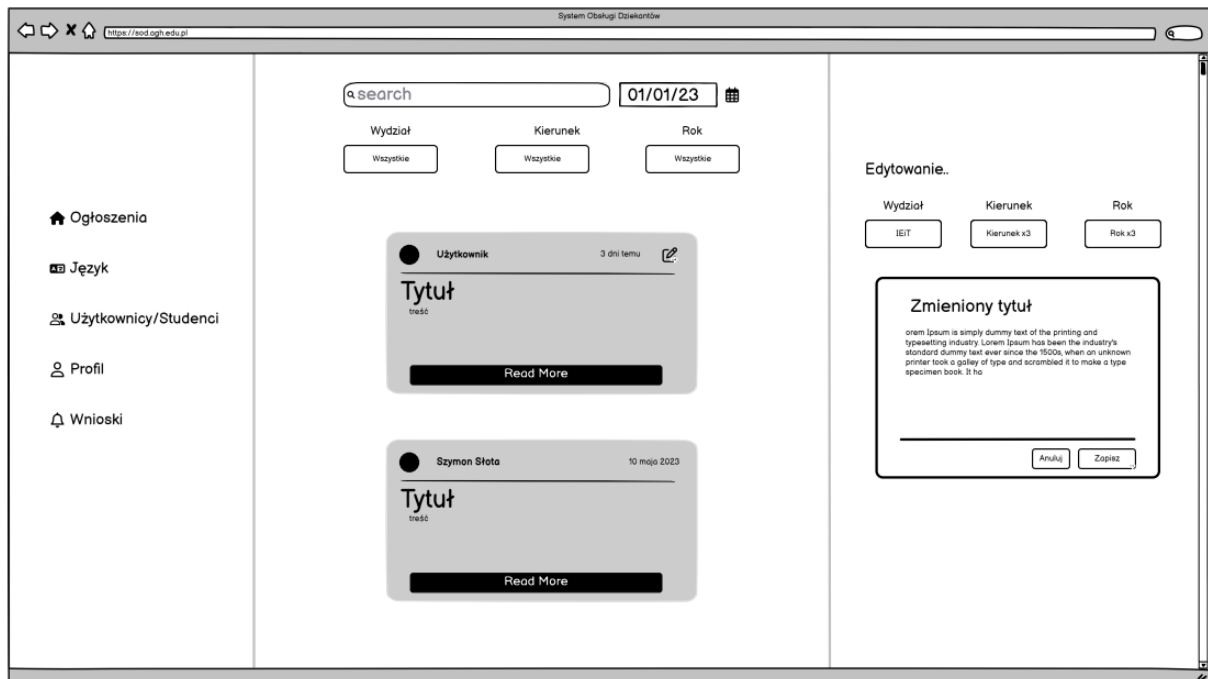
2.15: Makieta przedstawiająca posty pracowników

- Wstawienie ogłoszenia dla wszystkich studentów danego:
 - roku
 - wydziału
 - roku i wydziału
 - wydziału i kierunku
 - roku, wydziału i kierunku



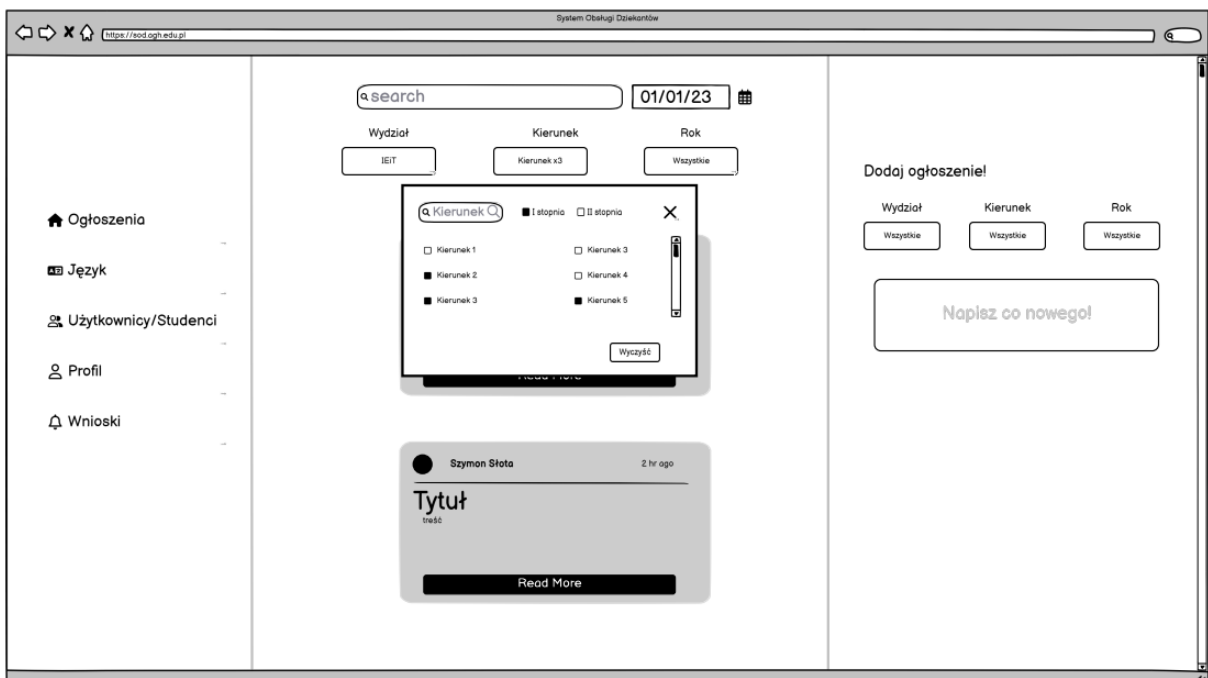
2.16: Makieta przedstawiająca dodawanie postu przez pracownika (prawa sekcja)

- Możliwość edycji dodanych postów (pojawia się jako nowy u studenta z flaga edited, stary znika)



2.17: Makieta przedstawiająca edytowanie postu pracownika

- Filtrowanie dodanych ogłoszeń



2.18: Makieta przedstawiająca filtrowanie postów

Wymagania niefunkcjonalne

- Aplikacja dostępna wyłącznie dla zalogowanych użytkowników
- Poprawne działanie w przeglądarkach Chrome, Firefox, EDGE
- Skalowanie aplikacji na różnych rozdzielczościach ekranu
- Obsługa grupy użytkowników - pracownicy dziekanatów AGH (Obsługa dużej liczby połączeń(10^3+), zapisów, oraz odczytów z bazy danych)
- Wyświetlanie aplikacji w dwóch językach: polskim i angielskim

2.3. Opis funkcji użytkowników

2.3.1. Rejestracja - aplikacja webowa

Jako: gość
Chcę: zarejestrować się w systemie
By: móc korzystać z aplikacji

Kryteria akceptacyjne:

1. Wyświetla się formularz rejestracji
2. Podanie danych personalnych
3. Podanie hasła
4. Potwierdzenie hasła
5. Jeśli podana została domena konta pracowniczego - @agh.edu.pl - użytkownik czeka na zatwierdzenie konta przez admina, w przypadku odrzucenia proces rejestracji zaczyna się od nowa.
6. Po potwierdzeniu, na maila pracownika przychodzi link do aktywacji konta, który przenosi użytkownika na stronę logowania. [7]

2.3.2. Rejestracja - aplikacja mobilna

Jako: gość
Chcę: zarejestrować się w systemie
By: móc korzystać z aplikacji

Kryteria akceptacyjne:

1. Wyświetla się formularz rejestracji
2. Podanie danych personalnych
3. Podanie hasła
4. Potwierdzenie hasła
5. Podanie wydziału
6. Podanie kierunku
7. Podanie roku
8. Zarejestrowanie się przenosi użytkownika na stronę logowania

2.3.3. Logowanie

Jako: wszyscy użytkownicy
Chcę: zalogować się
By: korzystać z aplikacji

Kryteria akceptacyjne:

1. Wyświetla się formularz logowania
2. Po udanym uwierzytelnianiu następuje przejście na stronę główną
3. Po nieudanym uwierzytelnianiu wyświetla się błąd o niepoprawnych danych logowania i użytkownik pozostaje na formularzu logowania

2.3.4. Wyświetlanie wpisów

Jako: wszyscy użytkownicy
Chcę: wyświetlić wpisy
By: przeglądać wątki

Kryteria akceptacyjne: —

2.3.5. Tworzenie wpisów

Jako: starosta/p. WRSS/pracownik/admin
Chcę: dodać post
By: poinformować odpowiednią grupę użytkowników

Kryteria akceptacyjne:

1. W głównym widoku aplikacji znajduje się formularz umożliwiający stworzenie nowego ogłoszenia
2. W formularzu znajdują się pola do wprowadzenia grupy docelowej, tytułu i treści ogłoszenia
3. Na dole formularza znajduje się przycisk umożliwiający dodanie uzupełnionego ogłoszenia

2.3.6. Edytowanie wpisów

Jako: starosta/p.WRSS/pracownik/admin
Chcę: edytować wybrany post
By: zaktualizować bądź go poprawić

Kryteria akceptacyjne:

1. W widoku głównym obok powiadomienia może znajdować się przycisk umożliwiający edytowanie
2. Po kliknięciu przycisku do edycji, dane z ogłoszenia wypełniają formularz umożliwiający dodanie ogłoszenia
3. W przypadku edycji formularz umożliwia zmianę zawartości zamiast dodawania nowego ogłoszenia
4. Po kliknięciu przycisku zapisz, zmiany w ogłoszeniu zostaną wprowadzone
5. Po kliknięciu przycisku anuluj, wszystkie zmiany zostaną wycofane

2.3.7. Filtrowanie zamieszczonych wpisów

Jako: wszyscy użytkownicy
Chcę: filtrować komunikaty
By: widzieć adekwatne dla moich potrzeb ogłoszenia

Kryteria akceptacyjne:

1. Nad listą ogłoszeń w widoku głównym znajdują się dostępne filtry
2. Można wybrać datę od do, wydziały, kierunki, lata, lub wpisać tekst
3. Każdy wybór filtruje listę ogłoszeń

2.3.8. Zmiana języka wyświetlania w aplikacji webowej

Jako: starosta/p. WRSS/pracownik/admin
Chcę: zmienić język wyświetlania
By: móc korzystać z aplikacji

Kryteria akceptacyjne:

1. Po lewej stronie w menu znajduje się przycisk języki
2. Po kliknięciu w przycisk pojawia się lista z dostępnymi językami aplikacji
3. Po kliknięciu w jedną z możliwości język aplikacji zmienia się

2.3.9. Zmiana języka wyświetlania w aplikacji mobilnej

Jako: student/starosta/p. WRSS
Chcę: zmienić język wyświetlania
By: móc korzystać z aplikacji

Kryteria akceptacyjne:

1. W pasku menu na dole aplikacji znajduje się przycisk ustawienia
2. Po kliknięciu na przycisk użytkownik zostaje przeniesiony do listy ustawień z dostępną opcją języki
3. Po kliknięciu na przycisk języki rozwija się lista z dostępnymi opcjami
4. Po kliknięciu na element listy język aplikacji zmienia się

2.3.10. Usuwanie wpisów w aplikacji webowej

Jako: starosta/p. WRSS/pracownik/admin
Chcę: usunąć post
By: uniemożliwić jego przeglądanie

Kryteria akceptacyjne:

1. W widoku głównym aplikacji obok wpisu może znajdować się przycisk do usunięcia go - zależy od tego, czy dany użytkownik ma uprawnienia do usunięcia danego ogłoszenia
2. Po kliknięciu na przycisk pojawia się okno do potwierdzenia operacji
3. Po zaakceptowaniu wpis zostaje usunięty

2.3.11. Nadawanie ról starostom

Jako: admin/p. WRSS
Chcę: nadać uprawnienia staroście
By: mógł publikować ogłoszenia

Kryteria akceptacyjne:

1. Użytkownik znajduje się w widoku listy studentów
2. Należy wyszukać odpowiedniego studenta za pomocą filtrów i wyszukiwania
3. Przy każdym studencie znajduje się rozwijane menu pozwalające nadać rolę - wybrać starosta
4. Zatwierdzić zmiany

2.3.12. Usuwanie uprawnień starosty

Jako:p.WRSS

Chcę:usunąć studentowi rolę starosty

By:zaktualizować pulę starostów

Kryteria akceptacyjne:

1. Użytkownik znajduje się w widoku listy studentów
2. Należy wyszukać odpowiedniego studenta za pomocą filtrów i wyszukiwania
3. Przy każdym studencie znajduje się rozwijane menu pozwalające nadać rolę - wybrać student
4. Zatwierdzić zmiany

2.3.13. Nadawanie uprawnień p. WRSS

Jako: admin

Chcę: nadać uprawnienia p. WRSS

By: mógł publikować ogłoszenia i mianować starostę

Kryteria akceptacyjne:

1. Użytkownik znajduje się w widoku listy studentów
2. Należy wyszukać odpowiedniego studenta za pomocą filtrów i wyszukiwania
3. Przy każdym studencie znajduje się rozwijane menu pozwalające nadać rolę - wybrać p. WRSS
4. Zatwierdzić zmiany

2.3.14. Usuwanie uprawnień p.WRSS

Jako:admin

Chcę:usunąć studentowi rolę przewodniczącego WRSS

By:zaktualizować pulę starostów

Kryteria akceptacyjne

1. Użytkownik znajduje się w widoku listy studentów
2. Należy wyszukać odpowiedniego studenta za pomocą filtrów i wyszukiwania
3. Przy każdym studencie znajduje się rozwijane menu pozwalające nadać rolę - wybrać student
4. Zatwierdzić zmiany

2.3.15. Usuwanie kont użytkowników

Jako: admin

Chcę: Móc usuwać konta użytkowników

By: zablokować dostęp osobom nieupoważnionym

Kryteria akceptacyjne:

- Pojedynczy użytkownik:
 1. Wyszukać docelowego użytkownika na liście
 2. Użyć przycisku usuń, aby go usunąć
 3. Potwierdzić usunięcie
- Wielu użytkowników:
 1. Wybrać docelowych użytkowników poprzez checkboxy przy ich danych na liście
 2. Użyć przycisku usuń
 3. Potwierdzić usunięcie wielu użytkowników

2.3.16. Zatwierdzanie konta pracownika

Jako: admin

Chcę: zatwierdzić wniosek o utworzenie nowego konta pracowniczego

By: pracownik mógł korzystać z aplikacji

Kryteria akceptacyjne:

1. W widoku powiadomień znajduje się komunikat o wniosku o zatwierdzenie nowego konta pracownika.
2. W komunikacie znajdują się przyciski pozwalające zaakceptować lub odrzucić wniosek.
3. Po zaakceptowaniu lub odrzuceniu wniosku w komunikacie widnieje informacja o podjętym wyborze.

2.3.17. Dodawanie wydziałów

Jako: admin

Chcę: dodać wydział

By: struktura uczelni w systemie odpowiadała tej prawdziwej

Kryteria akceptacyjne:

1. W widoku wydziały i kierunki znajduje się lista wydziałów.
2. Żeby dodać wydział należy kliknąć na przycisk oznaczony plusem i podać nazwę.
3. Po dodaniu wydziału należy dodać co najmniej jeden kierunek.
4. Użytkownik potwierdza modyfikację.

2.3.18. Dodawanie kierunków

Jako: admin

Chcę: dodać kierunek

By: struktura uczelni w systemie odpowiadała tej prawdziwej

Kryteria akceptacyjne

1. W widoku wydziały i kierunki znajduje się lista wydziałów.
2. Żeby dodać kierunek należy najpierw wybrać wydział do którego zostanie przypisany
3. Po kliknięciu na wydział z listy pojawi się lista kierunków, która jest przypisana do tego wydziału
4. Nad listą kierunków znajduje się przycisk oznaczony plusem, należy kliknąć na niego i wprowadzić nazwę kierunku
5. Użytkownik potwierdza modyfikację

2.3.19. Usuwanie wydziałów

Jako: admin

Chcę: usuwać wydział

By: struktura uczelni w systemie odpowiadała tej prawdziwej

Kryteria akceptacyjne:

1. Obok każdego wydziału na liście znajduje się opcja usunięcia go
2. Pojawia się okno potwierdzenia usunięcia
3. Po potwierdzeniu wydział zostaje usunięty

2.3.20. Usuwanie kierunków

Jako: admin

Chcę: usuwać kierunek

By: struktura uczelni w systemie odpowiadała tej prawdziwej

Kryteria akceptacyjne:

1. W widoku wydziały i kierunki po wybraniu wydziału, obok każdego kierunku na liście znajduje się opcja usunięcia go
2. Pojawia się okno potwierdzenia usunięcia
3. Po potwierdzeniu kierunek zostaje usunięty

2.3.21. Edytowanie wydziałów

Jako: admin

Chcę: modyfikować listy wydziałów

By: struktura uczelni w systemie odpowiadała tej prawdziwej

Kryteria akceptacyjne:

1. W widoku wydziały i kierunki obok każdego wydziału na liście znajduje się opcja modyfikacji
2. Po kliknięciu użytkownik może zmienić nazwę
3. Użytkownik wpisuje nową nazwę wydziału
4. Użytkownik potwierdza modyfikację wydziału

2.3.22. Edytowanie kierunków

Jako: admin

Chcę: modyfikować listy kierunków

By: struktura uczelni w systemie odpowiadała tej prawdziwej

Kryteria akceptacyjne:

1. Obok każdego kierunku na liście znajduje się opcja modyfikacji go, po kliknięciu użytkownik będzie mógł zmienić nazwę
2. Użytkownik wpisuje nową nazwę kierunku
3. Użytkownik potwierdza modyfikację kierunku

2.3.23. Edycja profilu

Jako: Wszyscy

Chcę: Edytować swój profil

By: Zaktualizować dane

Kryteria akceptacyjne:

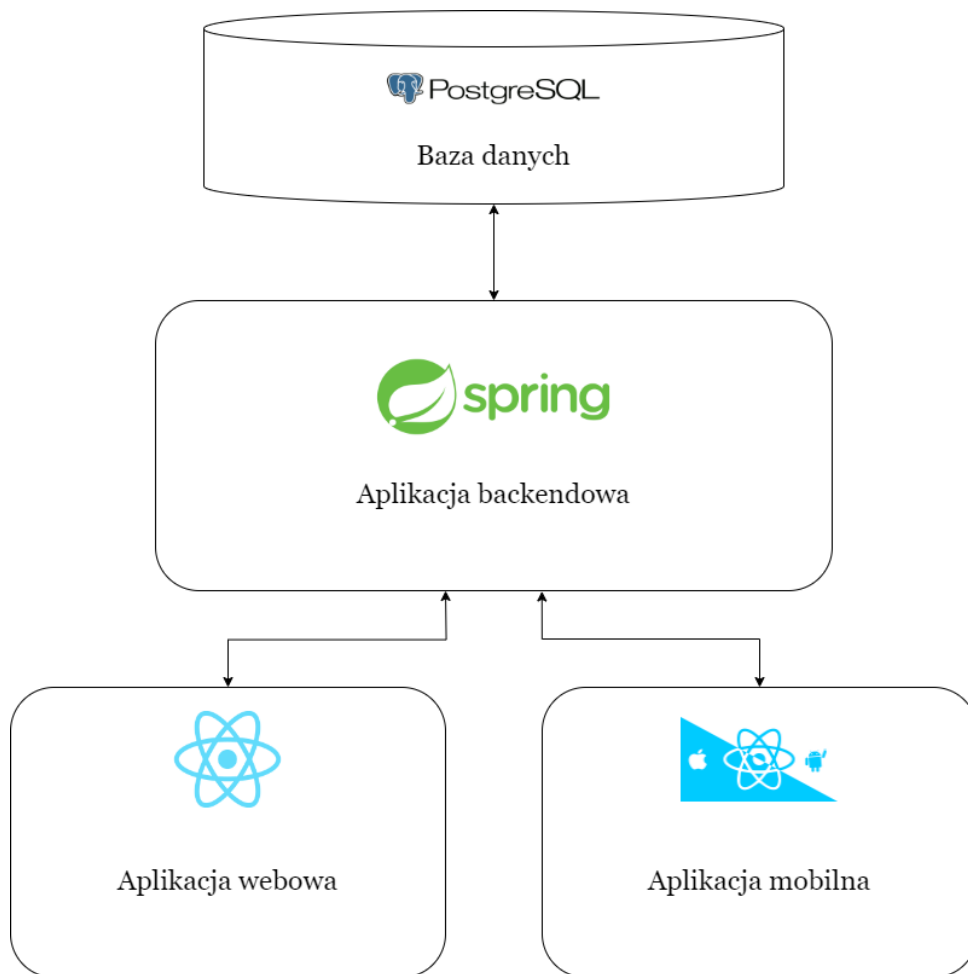
1. Jestem zalogowany w widoku profil użytkownika
2. Przechodzę do edycji profilu po użyciu przycisku
3. Wypełniam formularz, dzięki któremu zaktualizuje dane
4. Po akceptacji automatycznie przechodzę na profil

Rozdział 3

Wybrane aspekty realizacji

3.1. Ogólny opis projektu

Powstały system opiera się na relacyjnej bazie danych PostgreSQL [10] i 3 aplikacjach:



3.1: Architektura systemu

- aplikacja backendowa w technologii Java Spring [19] - serwis operujący na danych z bazy, udostępniający je pozostałym aplikacjom
- aplikacja webowa stworzona przy pomocy React [11] - aplikacja służąca do komunikacji pracowników dziekanatu ze studentami
- aplikacja mobilna napisana w React Native [12] - aplikacja służąca do przeglądania ogłoszeń przez studentów oraz do wysyłania komunikatów przez starostów i przewodniczących WRSS

3.2. Stos technologiczny

3.2.1. Backend

PostgreSQL

Projektując bazę danych dla naszej aplikacji zdecydowaliśmy się na wykorzystanie PostgreSQL. Niezawodność i stabilność silnika dają pewność, że dane będą bezpieczne i dostępne nawet w przypadku dużego obciążenia. To kluczowa cecha dla naszego projektu, w którym wysoka wydajność jest priorytetem. Ze względu na relacyjność i transakcyjność wybranej bazy, możliwe jest obsługiwanie zaawansowanych typów danych, umożliwiające złożone zapytania, co daje dużą elastyczność w wykonywanych operacjach, co jest niezbędne przy budowie naszej aplikacji.

Spring

Do stworzenia backendu naszej aplikacji zdecydowaliśmy się użyć Javy w połączeniu ze Spring Framework. Spring oferuje narzędzia, które ułatwiają zarządzanie zależnościami, tworzenie aplikacji webowych, mikroservisów, operowanie na danych oraz zapewnianie bezpieczeństwa. To nie tylko ogólna platforma, ale zestaw konkretnych modułów takich jak Spring Boot, Spring Data [20] czy Spring Security [18], które rozwiązują konkretne problemy związane z tworzeniem aplikacji w Javie.

Hibernate

Hibernate [6] to framework ORM (Object-Relational Mapping). Jego główną zaletą, ze względu na którą zdecydowaliśmy się z niego skorzystać, jest możliwość mapowania obiektów na tabele w bazie danych oraz operowanie na nich za pomocą języka Java. To sprawia, że praca z danymi staje się bardziej obiektowa i wygodniejsza.

Jedną z kluczowych zalet Hibernate jest eliminacja potrzeby pisania złożonych zapytań SQL. Zamiast tego, można operować na obiektach, co sprawia, że kod staje się bardziej czytelny i łatwiejszy w utrzymaniu. Mapowanie relacyjno-obiektowe zapewnia również wygodne zarządzanie relacjami między tabelami.

Hibernate obsługuje transakcyjność, co gwarantuje integralność danych. Dzięki temu mechanizmowi, można wykonywać operacje na bazie danych w sposób atomowy.

3.2.2. Rozdzielenie aplikacji mobilnej i webowej

Decyzja o użyciu różnych technologii umożliwiła nam dostosowanie każdej z aplikacji do specyficznych wymagań środowiskowych. Aplikacja mobilna, napisana w React Native, została zoptymalizowana pod kątem funkcji urządzeń przenośnych, co umożliwiło lepsze wykorzystanie możliwości mobilnych. Z kolei aplikacja webowa w Reactie była dostosowana do przeglądarek internetowych, zapewniając użytkownikom płynne i efektywne doświadczenie online. Dzięki tej strategii zyskaliśmy elastyczność w dostosowaniu obu aplikacji do unikalnych potrzeb ich docelowych środowisk, co przekłada się na lepszą jakość i wydajność całego systemu.

Dodatkowo, decyzja o użyciu różnych technologii wynikała z troski o skalowalność projektu. Tworzenie jednej ogromnej aplikacji w React Native mogłoby prowadzić do znacznego wzrostu rozmiaru kodu, co utrudniłoby zarządzanie nim, a także wpłynęłoby negatywnie na procesy debugowania i testowania. Dzięki podziałowi na dwie aplikacje zminimalizowaliśmy potencjalne problemy związane z zarządzaniem dużym kodem źródłowym, co przyczyniło się do efektywniejszego procesu rozwoju, utrzymania i testowania.

3.2.3. Aplikacja webowa

React

W celu stworzenia aplikacji webowej skorzystaliśmy z frameworka React, w szczególności wybierając środowisko Vite.js oraz bibliotekę stylów Tailwind CSS.

Vite.js [23] to nowoczesne narzędzie do budowy aplikacji webowych, które oferuje szybkie czasy budowania oraz rozwijania projektu. Dzięki Vite.js uzyskaliśmy efektywną konfigurację projektu, co wpłynęło pozytywnie na wydajność naszej aplikacji.

Tailwind [22] CSS to narzędzie do budowania interfejsów użytkownika oparte na klasach, co umożliwiło nam szybkie i konsekwentne tworzenie stylów. Tailwind dostarcza predefiniowanych klas, co przyspieszyło proces projektowania i pozwoliło utrzymać spójność wizualną.

Vite.js

Wdrożenie Vite.js jako środowiska projektowego było kluczowe dla naszej aplikacji webowej. Vite.js dostarcza szybką reakcję na zmiany w kodzie, co znacznie ułatwia proces rozwijania i testowania aplikacji. Jego nowoczesna architektura sprawiła, że projektowanie aplikacji webowej stało się bardziej efektywne.

Tailwind CSS

Tailwind CSS był integralną częścią naszego stosu technologicznego do stylizacji aplikacji. Dzięki prostocie użycia klas i gotowym komponentom, Tailwind pozwolił nam skupić się na projektowaniu interfejsu bez konieczności dużego nakładu pracy przy pisaniu niestandardowych stylów CSS.

3.2.4. Aplikacja mobilna

React Native

W celu stworzenia aplikacji mobilnej wykorzystaliśmy framework React Native. Wybraliśmy takie rozwiązanie przede wszystkim ze względu na uniwersalność kodu - aplikacje pisane w React Native mogą być uruchamiane zarówno w systemie Android jak i iOS. React Native oferuje szybkość i niezawodność, a także pozwala na bezproblemowe rozwijanie aplikacji, dzięki licznym wbudowanym narzędziom i natywnym modułom.

SecureStore

Warto wspomnieć o tym module, jako że wpłynął znacząco na ogólne funkcjonowanie naszej aplikacji. SecureStore [3] pozwala na przetrzymywanie danych w pamięci urządzenia, np tokenów lub zapisanych preferencji. Dzięki niemu, można inteligentnie przechowywać język aplikacji lub motyw, nawet po wylogowaniu - niezależnie od zalogowanego użytkownika.

Expo CLI

Expo CLI [4] to narzędzie firmy Expo stworzone w celu ułatwienia i przyspieszenia konfiguracji projektów aplikacji mobilnych.

Expo Go

Znaczącym dodatkiem od firmy Expo jest aplikacja Expo Go [5], pozwalająca na uruchomienie aplikacji w trybie developerskim na każdym urządzeniu z systemem Android lub iOS, co ułatwia przedstawienie demo klientowi oraz prowadzenie testów w zespole - cały proces tworzenia produktu jest pod kontrolą.

3.2.5. Podsumowanie

Podsumowując, nasze rozwiązanie opiera się na solidnym fundamencie technologicznym, obejmującym różnorodne narzędzia i frameworki. W obszarze backendu wybraliśmy PostgreSQL jako bazę danych, gwarantującą niezawodność, stabilność i wysoką wydajność nawet przy dużym obciążeniu. W połączeniu z Java i Spring Framework, nasz backend jest nie tylko bezpieczny, ale też elastyczny dzięki możliwości obsługi zaawansowanych typów danych i złożonych zapytań.

Aplikacja webowa oparta na React, Vite.js i Tailwind CSS to efektywne połączenie nowoczesnych narzędzi, umożliwiających szybki rozwój, testowanie i utrzymanie spójnego interfejsu użytkownika. Vite.js zapewnia szybkie reakcje na zmiany w kodzie, podczas gdy Tailwind CSS ułatwia konsekwentne tworzenie stylów.

Dla aplikacji mobilnej skorzystaliśmy z React Native, co pozwoliło nam osiągnąć uniwersalność kodu i szybkość działania na platformach Android i iOS. Dodatkowo, wykorzystanie narzędzi takich jak SecureStore, Expo CLI i Expo Go umożliwiło nam nie tylko bezpieczne przechowywanie danych, ale także szybką konfigurację projektu oraz łatwe uruchamianie aplikacji w trybie developerskim na różnych urządzeniach.

3.3. Struktura systemu - Backend

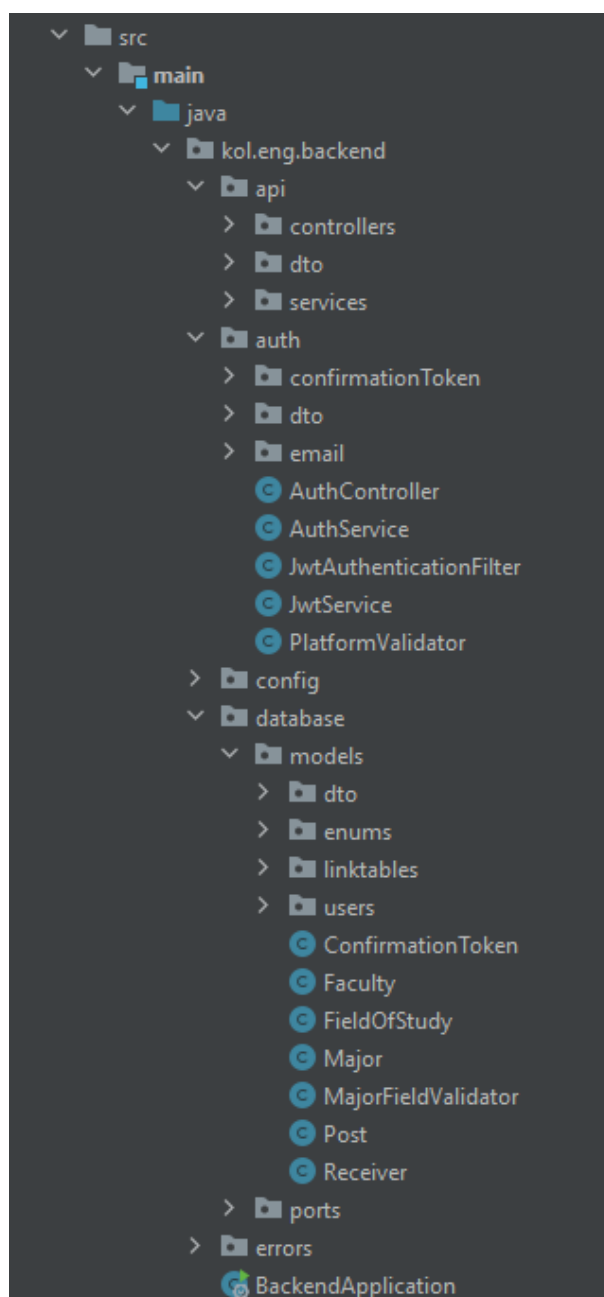
Projekt został stworzony według modelu REST - architektury składającej się z warstw prezentacji, logiki biznesowej i warstwy danych. Zastosowanie tego modelu pozwoliło klarownie rozdzielić odpowiedzialności w aplikacji, umożliwiło segregację kodu zgodnie z jego funkcjonalnością. To podejście pozwoliło nam elastycznie rozbudowywać aplikację zachowując czytelność struktury kodu.



3.2: Wysokopoziomowa struktura aplikacji

Projektując aplikację, zdecydowaliśmy się stworzyć warstwę repozytoriów pośredniczącą między modelem danych a bazą danych. Ich głównym zadaniem jest obsługa wszelkich operacji na danych oraz izolacja logiki biznesowej od konkretnych mechanizmów dostępu do danych. Stanowią one spójną warstwę, która umożliwia zarządzanie operacjami odczytu, zapisu, modyfikacji i usuwania danych. Sprawiają, że kod staje się bardziej modułarny i elastyczny, co bardzo ułatwia ewentualne zmiany w strukturze danych lub sposobie ich przechowywania, bez wpływu na główną logikę aplikacji.

Usługi serwisu obsługiwane są przez warstwę kontrolerów i serwisów. Pierwsza z nich pozwala na dostęp do odpowiednich części logiki biznesowej. Wysyłając zapytanie do wystawionych endpoint'ów, wywołujemy odpowiednie metody serwisów drugiej wspomnianej warstwy. Tam, w sposób odizolowany, wykonują się wszystkie operacje na danych.



Rysunek 3.3: Struktura warstwowa projektu aplikacji

3.3.1. Udostępniane usługi

Poniżej opisane zostały wszystkie usługi udostępnione w zbudowanym systemie:

/posts - ogłoszenia

Kontroler PostController udostępnia:

- **/posts [POST]** - dodawanie postów
- **/posts/all-filtered [POST]** - przeglądanie wszystkich przefiltrowanych postów - endpoint, z którego korzystają pracownicy
- **/posts/edit [PATCH]** - edycja ogłoszeń
- **/posts/for-user [POST]** - przeglądanie postów skierowanych dla odpowiedniego studenta
- **/posts/owned-by-user [GET]** - przeglądanie komunikatów dodanych przez obecnie zalogowanego użytkownika
- **/posts/postId [DELETE]** - usuwanie postów

/syllabus - sylabus AGH

Kontroler SyllabusController udziela usług:

- **/syllabus/faculties [GET]** - dostęp do listy wydziałów
- **/syllabus/faculties [DELETE]** - usunięcie wydziałów
- **/syllabus/faculties/majors [GET]** - pobranie listy kierunków dla podanych wydziałów
- **/syllabus/faculty [POST]** - dodanie wydziału
- **/syllabus/faculty [DELETE]** - usunięcie wydziału
- **/syllabus/faculty [PATCH]** - edycja wydziału
- **/syllabus/major [POST]** - dodanie kierunku
- **/syllabus/major [DELETE]** - usunięcie kierunku
- **/syllabus/major/update [PATCH]** - edycja wydziału
- **/syllabus/majors [POST]** - tworzenie kierunków dla wydziału
- **/syllabus/majors [DELETE]** - usunięcie kierunków wydziału

/user - użytkownicy

Endpointy kontrolera UserController:

- **/user [GET]** - pobierz informacje o aktualnie zalogowanym użytkowniku
- **/user [DELETE]** - usuń konto użytkownika
- **/user/delete-many [DELETE]** - usuń wiele kont użytkowników
- **/user/edit-info [PATCH]** - edytuj informacje o użytkowniku
- **/user/edit-role [PATCH]** - edytuj rolę użytkownika
- **/user/employees [GET]** - pobierz wszystkich pracowników
- **/user/employees [POST]** - pobierz informacje o przefiltrowanych pracownikach
- **/user/role [GET]** - dostęp do roli użytkownika
- **/user/students [GET]** - dostęp do informacji o wszystkich użytkownikach
- **/user/students [POST]** - informacje o przefiltrowanych użytkownikach
- **/user/user-info [GET]** - dostęp do informacji o wybranym użytkowniku

/api/v1/auth - użytkownicy

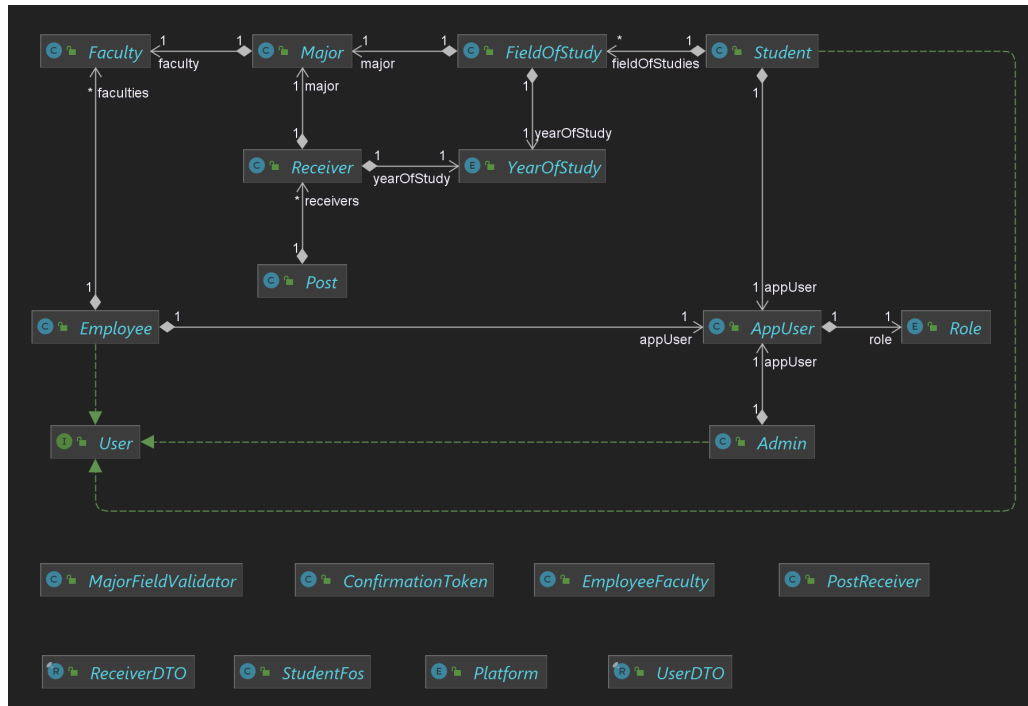
Kontroler AuthController udostępnia:

- **/api/v1/auth/authenticate [POST]** - logowanie na utworzone konto użytkownika
- **/api/v1/auth/confirm [POST]** - potwierdzenie konta pracownika
- **/api/v1/auth/register [POST]** - utworzenie konta w systemie

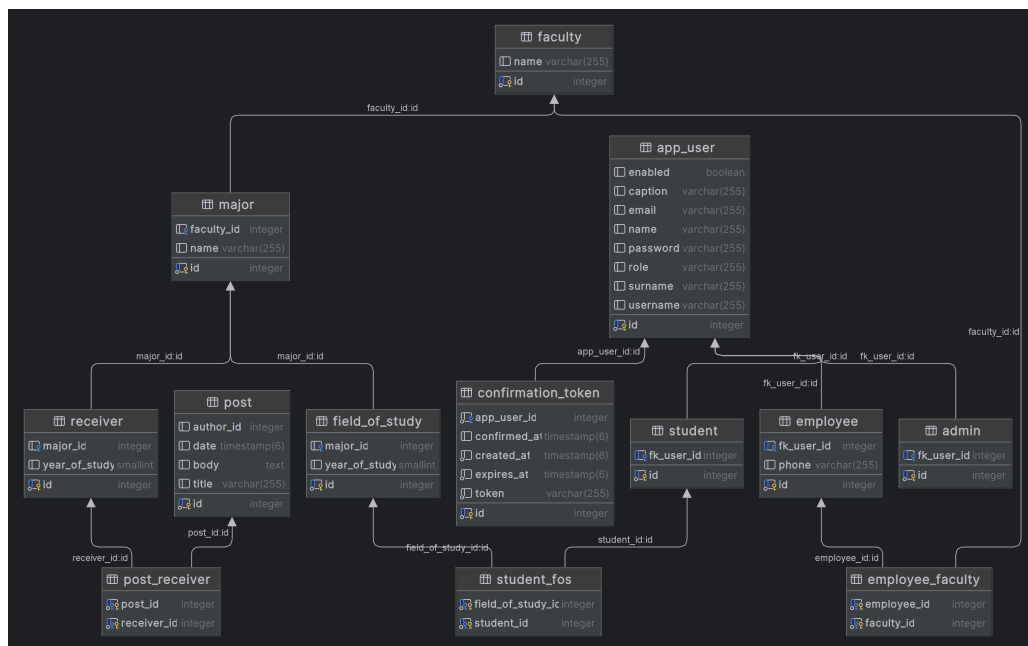
Szczegółowe informacje o każdym z wspomnianych endpointów znajdują się w dołączonym do pracy załączniku `praca-System_Obsługi_Dziekanatów_api_doc-Grzesik-Kozak-Mikuła-Słota.pdf`.

3.3.2. Model danych

Poniżej prezentowany jest model obiektowy, bezpośrednio mapowany na bazę danych. Ten model ilustruje zaprojektowaną strukturę klas oraz ich wzajemne relacje. Ukazuje, w jaki sposób zrealizowaliśmy powiązania między użytkownikami, strukturą uczelni a komunikatami w kontekście kodu.



3.4: Diagram klas



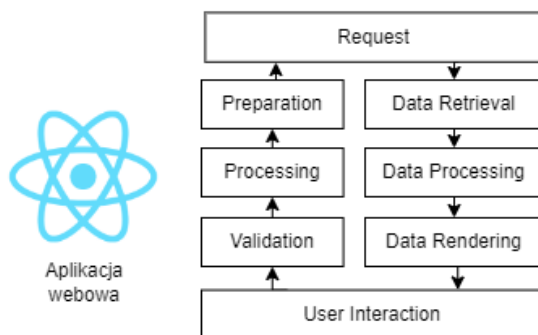
3.5: Baza relacyjna PostgreSQL

Tworząc schemat bazy danych staraliśmy się odwzorować rzeczywistą strukturę wydziałów, kierunków i lat, co zakłada, że jeden wydział zawiera wiele kierunków na przestrzeni 5 lat. W celu urzeczywistnienia tej idei, powstały odpowiednie tabele:

- **Tabela AppUser** - encja w bazie danych trzymająca wspólne dane każdego użytkownika w systemie - dane takie są konieczne, aby użytkownik prawidłowo działał w systemie
- **Tabela Admin/Employee/Student** - encja rozszerzająca domyślnego usera w dowolne dane, taka architektura zredukowała nam ilość pustych pól w bazie i umożliwiła łatwą skalowalność zarówno w pod kątem dodania nowych danych, trzymanych dla użytkownika, jak i dodanie całkiem nowych ról i użytkowników do naszego systemu
- **Tabela EmployeeFaculty** - encja umożliwiająca nam identyfikowanie z jakimi wydziałami jest połączony dany pracownik
- **Tabela Post** - encja trzymająca dane o poście bez powiązania go z jakimkolwiek odbiorcą
- **Tabela Receiver** - encja posiada informacje o odbiorcy dla postów, możemy w niej znaleźć rok studiów, kierunek oraz wydział odbiorcy postu ex. (1 rok, Informatyka, WI)
- **Tabela PostReceiver** - encja trzymająca powiązania między danym postem a odbiorcami, dla których jest on dedykowany
- **Tabela Major** - encja trzymająca nazwę kierunku i powiązanie do wydziału
- **Tabela Faculty** - encja trzymająca tylko nazwę wydziału, dzięki wydzieleniu tej informacji z tabeli Major, edytując wydział robimy to tylko w jednym miejscu a wszystkie połączone z nim kierunki mają aktualną wartość
- **Tabela FieldOfStudy** - encja pełni podobną rolę co Receiver, trzyma dane tego samego typu, ale redukuje nam skomplikowanie zależności, sprawia że relacje są bardziej przejrzyste oraz dzięki temu nasza architektura jest bardziej elastyczna - ułatwia możliwe modyfikacje i rozwój w przyszłości
- **Tabela StudentFos** - encja trzymająca powiązania między danym studentem a kierunkiem jego edukacji
- **Tabela ConfirmationToken** - encja trzymająca dane związane ze statusem konta pracownika, tabela służąca funkcjonowaniu mechanizmu aktywacji kont pracowniczych przez email

3.4. Struktura systemu - aplikacja webowa

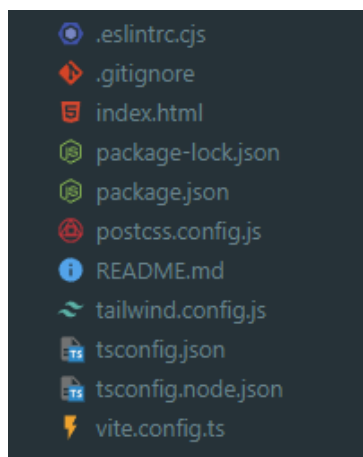
Aplikacja webowa została opracowana zgodnie z architekturą RESTową, a struktura projektu została starannie rozplanowana, aby precyzyjnie zorganizować kod.



3.6: Wysokopoziomowa struktura aplikacji webowej

Użytkownik inicjuje interakcję poprzez wysłanie żądania, które uruchamia proces pobierania danych z bazy. Następnie otrzymane informacje są przetwarzane, dostosowywane do czytelnej formy, i prezentowane w interfejsie użytkownika. Użytkownik ma możliwość interakcji z danymi, a wprowadzone informacje są poddawane walidacji. Po zatwierdzeniu, dane przechodzą przez dodatkowe procesy, a następnie są przygotowane do finalnej prezentacji, zanim odpowiedź zostanie zwrócona użytkownikowi w ramach pełnego cyklu żądania i przetwarzania.

3.4.1. Konfiguracja aplikacji



3.7: Pliki konfiguracyjne aplikacji webowej

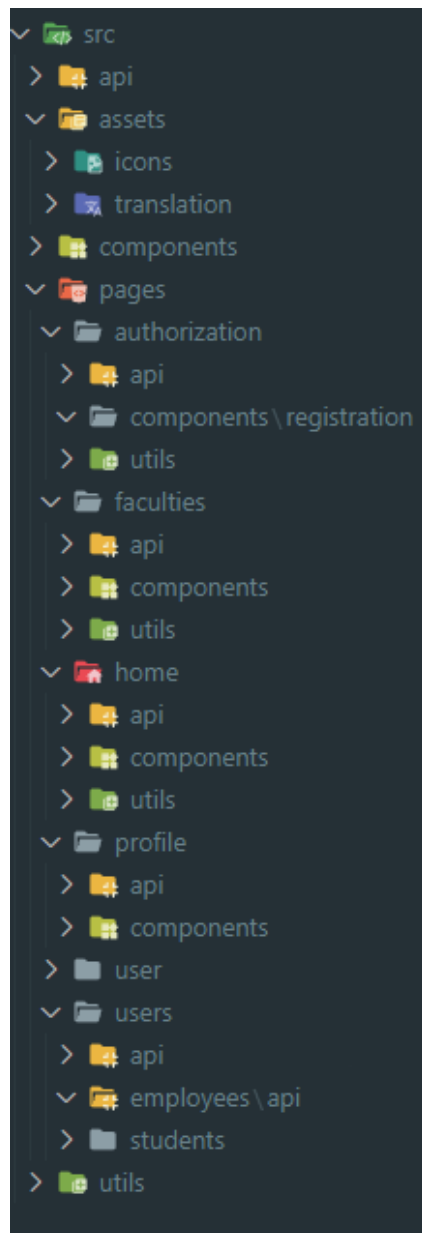
Style

W pliku **tailwind.config.js** zawarta jest kluczowa konfiguracja frameworka Tailwind CSS [22], która pełni istotną rolę w formowaniu wyglądu. W tym pliku znajduje się import do aplikacji predefiniowanych klas dostarczanych przez Tailwind CSS, które stanowią gotowe narzędzia do stylizacji elementów interfejsu. Jednak równie istotnym elementem są niestandardowe klasy, które samodzielnie definiujemy w celu precyzyjniejszej kontroli nad wyglądem poszczególnych elementów oraz umożliwiają nam stworzenie lepszej responsywności.

ESLint

Plik konfiguracyjny **.eslinttrc.cjs** [2] zawiera reguły analizy kodu, co pomaga przestrzegać standardów formatowania i eliminować potencjalne problemy.

3.4.2. Kod źródłowy



3.8: Struktura warstwowa projektu aplikacji webowej

Główny katalog projektu, o nazwie **src**, stanowi centralny punkt struktury aplikacji. Zawiera on kluczowe podkatalogi, które organizują kod źródłowy i zasoby projektu, umożliwiając jego czytelność i efektywne zarządzanie. Poniżej przedstawiamy główne podkatalogi:

- api
- assets
- components
- pages
- utils

Api

Katalog **api** pełni kluczową rolę w obszarze komunikacji z bazą danych. Znajdują się w nim pliki zawierające zapytania i operacje, które umożliwiają aplikacji dostęp do niezbędnych danych. Jest to główny podkatalog, dlatego też znajdziemy w nim zapytania, które są wykorzystywane w wielu komponentach bądź widokach, co sprzyja modularności i organizacji kodu.

Warto zauważyć, że w obrębie katalogu **api** nie tylko przechowujemy pliki z zapytaniami do bazy danych, ale także zajmujemy się obsługą błędów. W pliku o nazwie **errorHandler.tsx** skupiamy logikę dotyczącą zarządzania błędami, które mogą wystąpić podczas komunikacji z bazą danych czy innymi zasobami. Zabieg ten jest istotny dla zapewnienia spójności i niezawodności działania aplikacji, a także ułatwia śledzenie i debugowanie ewentualnych problemów.

Assets

Katalog **assets** pełni kluczową rolę w przechowywaniu różnorodnych zasobów niezbędnych dla poprawnego funkcjonowania interfejsu użytkownika. Oprócz tłumaczeń na języki polski i angielski, ten katalog gromadzi także ikony, stanowiące istotne elementy wizualne dla aplikacji.

```
export const loginComponent = {
  pl: {
    title: "Zaloguj się używając swojego konta",
    emailPlaceholder: "Wprowadź email",
    passwordPlaceholder: "Wprowadź hasło",
    button: "Zaloguj się",
    bottomText: "Nie masz konta?",
    bottomLink: "Zarejestruj się",
  },
  en: {
    title: "Sign in to your account",
    emailPlaceholder: "Enter email",
    passwordPlaceholder: "Enter password",
    button: "Sign in",
    bottomText: "No account?",
    bottomLink: "Sign up",
  },
};
```

3.9: Przykładowe tłumaczenie komponentu

Ważnym aspektem katalogu **assets** jest również zawarcie w nim typów wyliczeniowych, które zostały zharmonizowane z backendem. Przykłady takich typów mogą obejmować role użytkowników (*role*) czy lata studiów (*yearOfStudy*).

Unifikacja tych typów pomiędzy frontendem a backendem ma kluczowe znaczenie dla zachowania spójności danych w całej aplikacji. Dzięki temu rozwiązaniu, zarówno frontend, jak i backend korzystają z tych samych definicji, eliminując potencjalne niezgodności i ułatwiając rozwój i utrzymanie systemu.

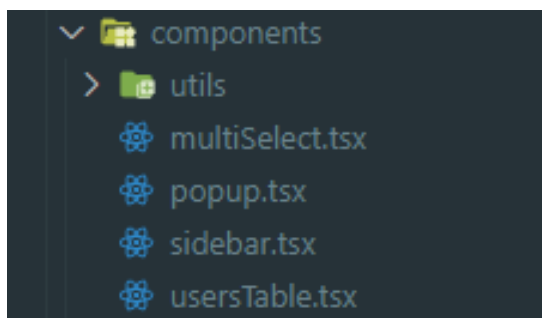
W praktyce, typy wyliczeniowe w katalogu **assets** stanowią centralne miejsce definiowania stałych, które są używane w różnych częściach aplikacji. Zastosowanie ich w jednym miejscu ułatwia późniejsze modyfikacje, a także zapewnia spójność w interpretacji tych samych danych zarówno po stronie klienta, jak i serwera.

Components

Katalog **components** stanowi centralne miejsce przechowujące globalne komponenty, które są wielokrotnie wykorzystywane w różnych obszarach aplikacji. Umieszczenie ich w tym katalogu przyczynia się do zwiększenia czytelności i efektywności zarządzania kodem źródłowym. Poniżej wymienione są niektóre z kluczowych komponentów znajdujących się w tym katalogu:

- **Multiselect**: Komponent umożliwiający użytkownikowi wybór wielu opcji z rozwijanej listy. Jego uniwersalność pozwala na używanie go w różnych częściach aplikacji, gdzie konieczny jest wybór z wielu możliwości.
- **Popup**: Komponent odpowiedzialny za wyświetlanie dynamicznych okienek, co stanowi istotny element interakcji z użytkownikiem. Może być wykorzystywany do pokazywania powiadomień, komunikatów czy formularzy.
- **UsersTable**: Komponent reprezentujący tabelę użytkowników. Tabela ta może być wykorzystywana w różnych miejscach, gdzie istnieje potrzeba prezentacji danych o użytkownikach w formie czytelnej i łatwej do przeglądania.

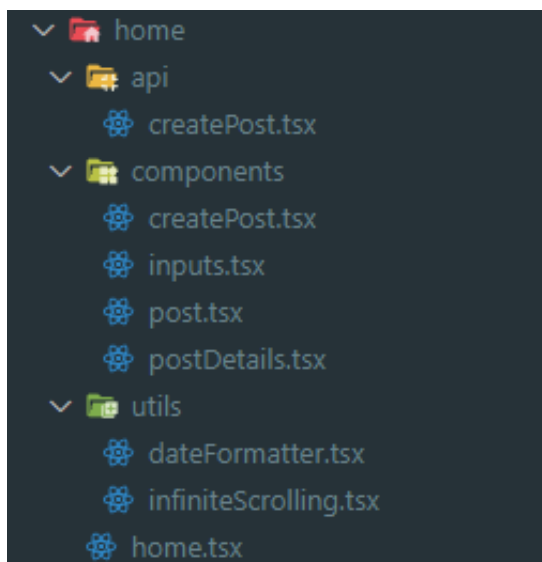
Dodatkowo, w tym katalogu znajduje się podkatalog **utils**, który zawiera metody specyficznie związane z obsługą wspomnianych wcześniej komponentów. Te narzędzia mają na celu ułatwienie operacji związanych z komponentami globalnymi, takich jak manipulacja danymi czy interakcje użytkownika. Ograniczenie tych narzędzi do **utils** pozwala na skoncentrowanie się na funkcjonalności komponentów w samym katalogu **components**, co sprzyja lepszej organizacji kodu.



3.10: Struktura podkatalogu components

Pages

W katalogu **pages** znajdują się główne widoki aplikacji, z których każdy skupia się na specyficznej funkcjonalności. Jest to najbardziej rozbudowany podkatalog, w którym używane są wszystkie główne podkatalogi, takie jak **assets**, **api**, **components**, **utils**, oraz także dedykowane podkatalogi dla niektórych stron.



3.11: Struktura katalogu home, który znajduje się w podkatalogu głównym pages

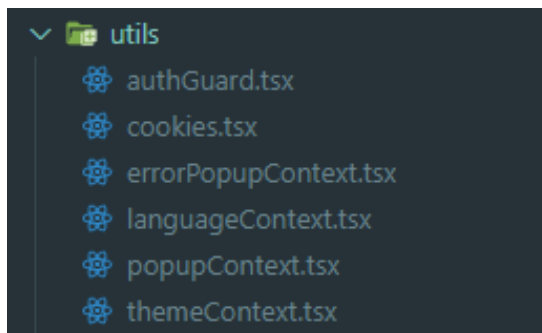
Poniżej przedstawiamy opis jednej z głównych stron, tj. **home**, jako przykładu:

W katalogu **pages/home** znajduje się implementacja widoku głównej strony aplikacji, czyli strony domowej. Strona ta jest jednym z najważniejszych punktów wejścia do aplikacji, integrując różne komponenty, narzędzia oraz logikę potrzebną do zapewnienia kompleksowej funkcjonalności.

- **api**: W przypadku głównej strony mamy dedykowany podkatalog **api**, zawierający pliki obsługujące zapytania tylko związane z tym konkretnym widokiem.
- **components**: Katalog **components** w **pages/home** zawiera komponenty specyficzne dla tego widoku. Są to np. **createPost.tsx**, **post.tsx**, odpowiednio pełnią funkcję sekcji do tworzenia postu oraz wyświetlania pojedynczego postu. Rozdzielenie komponentów wewnątrz tego katalogu pozwala na utrzymanie modularności i uniknięcie konfliktów nazw.
- **utils**: Katalog **utils** w **pages/home** zawiera narzędzia przeznaczone tylko do obsługi komponentów na stronie głównej. Są to funkcje pomocnicze, które są istotne tylko dla logiki związanej z tym konkretnym widokiem.
- **home.tsx**: Główny komponent (**home.tsx**) pełni rolę rodzica dla pozostałych komponentów. W tym pliku skupia się logika ogólna dla tego widoku, a także importuje i renderuje komponenty z katalogu **components** oraz korzysta z narzędzi z katalogu **utils**. To centralne miejsce, gdzie integrowane są wszystkie elementy strony domowej.

Utils

Katalog **utils** pełni kluczową rolę w przechowywaniu globalnych funkcji i narzędzi pomocniczych, które są szeroko wykorzystywane w różnych częściach aplikacji. Poniżej opisano niektóre z głównych plików znajdujących się w katalogu **utils**:



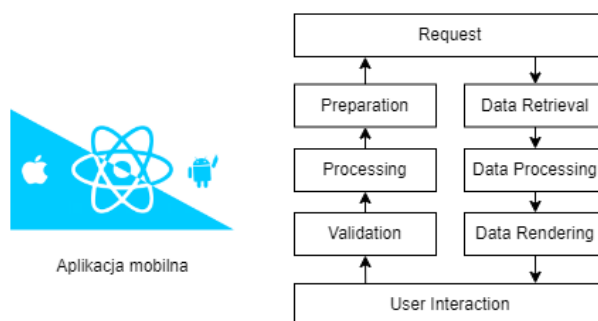
3.12: Podkatalog utils

- **themeContext.tsx**: Plik ten zawiera logikę obsługi kontekstu motywu (*themeContext*) w aplikacji. Kontekst ten jest odpowiedzialny za przechowywanie informacji o aktualnym motywie aplikacji oraz umożliwia zmianę motywu w trakcie działania aplikacji. Dzięki temu, aplikacja może dostosowywać swój wygląd do preferencji użytkownika.
- **languageContext.tsx**: Ten plik obsługuje kontekst języka (*languageContext*), który umożliwia dynamiczną zmianę języka interfejsu użytkownika. Dzięki temu, użytkownicy mają możliwość korzystania z aplikacji w wybranym przez siebie języku.
- **popupContext.tsx**: Plik zarządza kontekstem okienek (*popupContext*), kontrolując wyświetlanie i ukrywanie wyskakujących okienek w aplikacji. Ten kontekst jest istotny w obszarze komunikacji z użytkownikiem, umożliwiając dynamiczne pokazywanie ważnych informacji czy formularzy.
- **errorPopupContext.tsx**: Podobnie jak *popupContext*, ten plik obsługuje kontekst, jednak jest dedykowany do obsługi okienek błędów. Zapewnia strukturę umożliwiającą wyświetlanie informacji o błędach w sposób zorganizowany i przyjazny dla użytkownika.
- **cookies.tsx**: Plik ten jest odpowiedzialny za obsługę ciasteczek (*cookies*) w aplikacji. Zarządza operacjami związanymi z przechowywaniem, odczytywaniem i usuwaniem danych z ciasteczek, co jest istotne w przypadku przechowywania informacji o sesji użytkownika czy preferencjach.
- **authGuard.tsx**: Plik ten zawiera funkcje i logikę zabezpieczającą poruszanie się po aplikacji. Odpowiada za sprawdzanie autoryzacji użytkownika przed dostępem do określonych widoków. W przypadku braku autoryzacji, za pomocą metody z biblioteki *react-router-dom* [13] przekierowuje użytkownika do strony logowania.

Katalog **utils** pełni zatem funkcję magazynu narzędzi ogólnego zastosowania, które wspierają różne aspekty funkcjonalności aplikacji.

3.5. Struktura systemu - aplikacja mobilna

Tworzenie aplikacji mobilnej opierało się na zaprojektowaniu struktury komponentów zawierających część połączeniową [17], logiczną oraz stricte prezentacyjną. Ponieważ cały system opiera się o model RESTowy, część łącząca aplikację z backendem polega na wysyłaniu i obsłudze odpowiednich zapytań.



3.13: Wysokopoziomowa struktura aplikacji mobilnej

Interakcja użytkownika z aplikacją, np. wprowadzenie danych lub wykonanie akcji, przygotowuje i wysyła zapytanie na backend. Następnie z otrzymanej od serwera odpowiedzi odczytywane są dane, które aplikacja przetwarza i przygotowuje do wykorzystania w logice lub przedstawia je na ekranie użytkownika.

3.5.1. Konfiguracja aplikacji

```

> .expo
> .vscode
> assets
> node_modules
> src
⚙ .env
💎 .gitignore
JS App.js
{} app.json
B babel.config.js
🐳 Dockerfile
{} package-lock.json
{} package.json
  
```

3.14: Pliki konfiguracyjne aplikacji mobilnej

Nawigacja

W **App.js** znajduje się stos nawigacyjny aplikacji. Wykorzystuje on moduł **createNativeStackNavigator** pochodzący z biblioteki **@react-navigation/native-stack** [21] pozwalający na nawigację między ekranami.

Wspomniany stos nawigacyjny nie jest skomplikowany, jest to zestaw ekranów o tych samych priorytetach, z wyjątkiem widoku logowania, który zawsze uruchamiany zostaje jako pierwszy w celach weryfikacji tokenu oraz uruchamiania aplikacji po raz pierwszy.

Zmienne środowiskowe

W pliku konfiguracyjnym **.env** znajdują się modyfikacje pól konfigurujących sposób uruchamiania aplikacji.

Zmienne środowiskowe, które modyfikowaliśmy w celu udostępnienia testowego demo produktu to:

- **EXPO_PUBLIC_API_URL** - adres wraz z portem, na który wysyłane są requesty do backendu
- **REACT_NATIVE_PACKAGER_HOSTNAME** - adres wraz z portem, na którym udostępniana jest zbudowana i uruchomiona aplikacja

Biblioteki i zaimportowane moduły

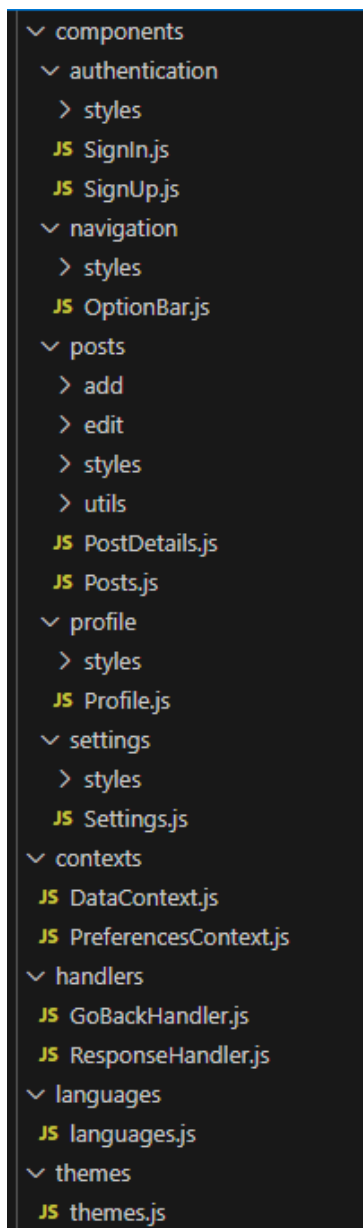
W projekcie używane jest wiele modułów pozwalających na usprawnienie interakcji użytkownika z aplikacją, zainstalować je można za pomocą komendy **npm install {...}** [9].

Znajdują się w katalogu **node_modules**.

Moduły, których użyliśmy w naszej aplikacji to:

- **@react-navigation/native** - moduł służy do budowania struktury nawigacyjnej, zarządzania ekranami i innymi elementami nawigacyjnymi
- **expo-secure-store** - moduł odpowiedzialny za asynchroniczne przechowywanie danych
- **@react-native-community/datetimepicker** - to pakiet społecznościowy dla React Native, który dostarcza komponenty do obsługi wyboru daty i czasu w aplikacjach mobilnych [1]
- **react-native-country-flag** - moduł umożliwia wyświetlanie flag państw w aplikacjach mobilnych [14]
- **react-native-element-dropdown** - moduł, który pozwala na wykorzystanie modyfikowalnego menu rozwijanego [15]
- **react-native-vector-icons** - moduł, pozwalający na wstawianie ikon - wykorzystaliśmy ikony Ionicons [16]

3.5.2. Kod źródłowy



3.15: Struktura warstwowa projektu aplikacji mobilnej - katalog **src**

W katalogu znajduje się kod źródłowy aplikacji. Na jego elementy składają się:

Komponenty

Komponenty to pliki w folderach **components**, pełniące rolę logiczną, połączeniową i prezentacyjną - w nich odbywa się logika modyfikacji i walidacja danych, z którymi miał interakcję użytkownik. Są to pliki o rozszerzeniu ".js" znajdujące się w katalogach odpowiadających za odpowiednie ekrany np.: "posts/Posts.js", "settings/Settings.js". Tam też wysyłane są zapytania do serwisu.

Lista grup komponentów aplikacji mobilnej:

- **authentication**

Komponent **SignUp** odpowiada za rejestrację a **SignIn** za logowanie w systemie.

- **navigation**

Komponent **OptionBar** odpowiada za nawigację po aplikacji.

- **posts**

W tej grupie znajduje się 5 komponentów:

- **Posts**, odpowiadający za wyświetlanie postów
- **PostDetails**, odpowiadający za wyświetlanie szczegółów postów
- **AddPost**, odpowiadający za funkcjonalność dodawania komunikatu
- **MyPosts**, jako komponent podglądu na opublikowane przez użytkownika posty
- **EditPost**, który odpowiada za edycję ogłoszenia

- **profile**

Komponent **Profile** odpowiada za wgląd na dane zalogowanego użytkownika oraz możliwość ich modyfikacji.

- **settings**

Komponent **Settings** to zbiór opcji i akcji jakie może podjąć użytkownik. Jest to na przykład zmiana języka, motywu, wylogowanie się lub usunięcie konta.

W odpowiednio nazwanych folderach odpowiedzialnych za dany ekran/komponent, np w **posts** znajdują się również pliki stylizacyjne, w przydzielonych im katalogach **styles**.

Folder **utils** przechowują dodatkowe funkcje, pozwalające np.: sformatować datę.

Wspomniane wcześniej modyfikacje danych odbywające się w odpowiednich komponentach polegają na przygotowaniu ich do prezentacji, przykładem jest połączenie kierunku, wydziału i roku w jeden element sylabusu, np. "Informatyka IV, WI", który następnie wyświetlany będzie na liście kierunków w profilu użytkownika.

Walidacja danych to głównie sprawdzenie zgodności wprowadzanych informacji przez użytkownika, przykładem sprawdzenia jest walidacja poprawności adresu email, którym rejestruje się użytkownik - musi on być w domenie **@student.agh.edu.pl**.

Konteksty

Konteksty to funkcje z katalogu **contexts** pozwalające wstrzyknąć globalne dane do komponentu, np. aktualny język, czy motyw, pobrane przy budowie aplikacji z modułu SecureStore.

Języki i motywy

Katalogi **languages** i **themes** zawierają pliki z odpowiednio tłumaczeniami i kolorami wykorzystywanymi w przy stylizacji ekranów, w zależności od preferencji użytkownika. Idea podstawiania odpowiedniego tłumaczenia lub koloru polega na wyborze odpowiedniego pola w zależności od ustawionego motywu lub języka np.:

```
const homeTranslations = {
  en : {
    title:"My posts",
    searchPlaceholder: "Search...",
    selectDate:"Select date",
    filterConfirm:"Filter",
    clearFilters:"Clear",
    emptyList:"There's no news... 😞",
    emptyListOfOwned:"Post something! 📝",
    connectionIssue:"Connection issue!"
  },
  pl : {
    title:"Moje posty",
    searchPlaceholder: "Szukaj...",
    selectDate:"Wybierz datę",
    filterConfirm:"Filtruj",
    clearFilters:"Wyczyść",
    emptyList:"Nie ma żadnych wiadomości... 😞",
    emptyListOfOwned:"Opublikuj coś! 📝",
    connectionIssue:"Problem z połączeniem!",
  }
}

const themes = {
  light:{
    backgroundMainPageColor:"#E4D5C7",
    postContainerColor:"#f7f6f5",
    textColor:"black",
  },
  dark:{
    backgroundMainPageColor:"#191E27",
    postContainerColor:"#1F3340",
    textColor:"gray",
  }
}
```

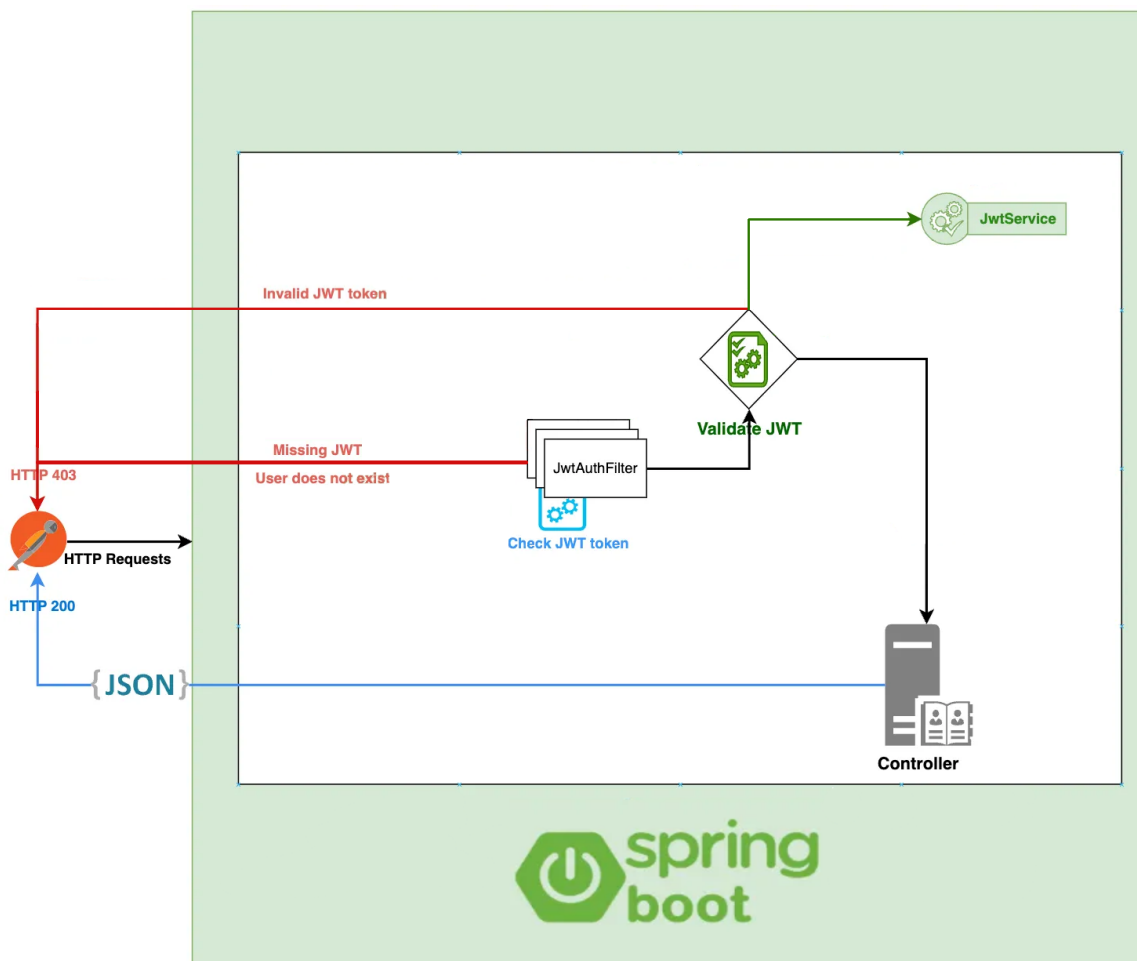
Rysunek 3.17: Przykład motywów

Rysunek 3.16: Tłumaczenia dla strony głównej

Obsługa błędów i akcji

W katalogu **handlers** znajdują się funkcje rozpoznające rodzaj błędu przychodzącego na podstawie kodu błędu z odpowiedzi na zapytanie, co zapobiega pojawianiu się błędów spowodowanych przez np. brak otrzymanych danych czy brak połączenia. Są również funkcje wykrywające używanie fizycznych przycisków telefonu, np przycisku powrót - jest to przydatne, założyliśmy na przykład, że nigdzie nie można "wrócić" ze strony głównej.

3.6. Bezpieczeństwo



Rysunek 3.18: Schemat działania security

Do autoryzacji i uwierzytelniania w naszym systemie wykorzystaliśmy JWT token [8]. JWT, czyli JSON Web Token, to kompaktowy sposób reprezentacji informacji między dwiema stronami. Jest to token, który zawiera zazwyczaj trzy sekcje:

- Nagłówek (Header): Zawiera informacje o typie tokenu (JWT) i używanym algorytmie szyfrowania, np. HMAC SHA256 czy RSA.
- Dane (Payload): Zawiera zazwyczaj informacje, takie jak dane użytkownika czy inne metadane. To sekcja, która stanowi użyteczne informacje przesyłane między stronami.
- Sygnatura (Signature): Tworzona jest na podstawie nagłówka, danych oraz tajnego klucza. Służy do weryfikacji, czy token nie został zmieniony podczas przesyłania.

JWT jest często wykorzystywany w kontekście uwierzytelniania i autoryzacji w aplikacjach webowych. Jest przenośny, bezstanowy i zawarte w nim informacje umożliwiają weryfikację tożsamości użytkownika. Ponadto, dzięki temu, że jest podpisany, strony mogą zweryfikować, czy token nie został sfałszowany. JWT jest powszechnie stosowany w architekturze RESTful API do przekazywania informacji uwierzytelniających między klientem a serwerem, stąd nasz wybór takiego, optymalnego i bezpiecznego, rozwiązania.

Powyższy diagram przedstawia schemat działania naszego REST API w oparciu o token JWT, odbywa się to w sposób następujący:

1. Każdy request, który przychodzi do backendu, powinien zawierać taki token, jeżeli tak nie jest, JwtAuthFilter automatycznie zwróci informację o błędzie.
2. Jeżeli token istnieje, Spring, poprzez UserDetailsService, pobierze z bazy danych informacje o użytkowniku przypisanym do otrzymanego tokenu. Na schemacie baza danych kryje się pod kontenerem dockerowym. JwtAuthFilter, poprzez JwtService, sprawdzi czy jest to ten sam użytkownik, o którym informację przechowuje token. W przeciwnym wypadku, JwtAuthFilter zwróci informację o błędnym tokenie.
3. Gdy token jest prawidłowy, następuje aktualizacja SecurityContextHolder, w którym na czas trwania przetwarzania danego requesta zostaje zapisana informacja, że użytkownik wykonujący zapytanie jest już zweryfikowany.
4. Po zapisaniu tej informacji DispatcherServlet odpakowuje rządanie użytkownika i jest ono przetwarzane.

3.7. Budowanie i wdrażanie

Przygotowując demo aplikacji dla klienta po uprzednim przygotowaniu wymaganych, odpowiednich plików Dockerfile, wygenerowaliśmy obrazy backendu, oraz obu frontedów. Następnie stworzyliśmy plik Docker-Compose, zawierający kontenery:

- backend
- frontend-web
- frontend-mobile

Plik ten umożliwia za pomocą jednej komendy uruchomić całą aplikację w Dockerze.

Aby jeszcze bardziej ułatwić proces, stworzyliśmy również skrypt w shellu, który pozwala aktywować całe środowisko aplikacyjne jednym kliknięciem.

Jego wywołanie:

1. uruchamia dodatkowy skrypt, podstawiający adres IP urządzenia z uruchomionym środowiskiem pod zmienne środowiskowe aplikacji mobilnej, aby można było w celu testowania uruchomić telefonem aplikację za pomocą Expo Go oraz mieć dostęp do backendu.
2. sprawdza czy obecny jest plik .tar z obrazami
3. sprawdza czy Docker jest zainstalowany i działa
4. importuje obrazy dockerowe
5. uruchamia kontenery

Tak przygotowany pakiet plików przesłaliśmy klientowi do przeprowadzenia testów, a jedynym wymogiem do uruchomienia aplikacji było posiadanie zainstalowanego środowiska Docker oraz odpowiedniej ilości dostępnego miejsca na dysku. Tak skonfigurowane demo znacznie ułatwiło klientowi testowanie naszej aplikacji.

3.8. Testowanie funkcjonalności oraz jakości kodu

W procesie rozwoju naszej aplikacji, testowanie opierało się głównie na podejściu manualnym. Nie stworzyliśmy kompletnego zestawu testów jednostkowych, jednak wprowadziliśmy pewne testy skupiające się na kluczowych funkcjach oraz istotnych komponentach aplikacji. Te testy miały na celu weryfikację podstawowych zachowań poszczególnych modułów, co umożliwiało nam ocenę ich poprawności działania na wczesnym etapie rozwoju.

Testując przyjęliśmy podejście skoncentrowane na wykorzystaniu scenariuszy użycia opisanych w rozdziale 2. W procesie testowania interfejsów API korzystaliśmy głównie z narzędzi takich jak Postman. Manualne działania obejmowały nie tylko ręczne testowanie, ale także śledzenie logów oraz wnikliwą weryfikację działania poszczególnych funkcji i modułów. Przez to podejście, mogliśmy dokładnie analizować i zapewniać poprawność interakcji pomiędzy różnymi częściami systemu.

Nasze testy polegały na odtworzeniu sytuacji, jakie mogą wystąpić podczas korzystania z aplikacji przez użytkowników. Manualne procedury testowania oparte na scenariuszach użycia, stanowiły solidną bazę dla wczesnej weryfikacji poprawności działania kluczowych komponentów aplikacji oraz symulację rzeczywistych interakcji z aplikacją.

Ta praktyka pozwoliła nam na szybką reakcję na ewentualne problemy oraz umożliwiła ocenę stabilności poszczególnych fragmentów kodu. Dzięki temu mogliśmy ocenić responsywność oraz zachowanie się aplikacji w różnych warunkach.

Rozdział 4

Organizacja pracy

4.1. Charakterystyka projektu i sposób jego realizacji

Nasz projekt, będący projektem rozwojowym, cechował się wyraźnie określonymi wymaganiami już na etapie początkowym. Miało to kluczowe znaczenie, ponieważ mogliśmy skupić się na określonych celach od samego startu. Dzięki temu uniknęliśmy niejasności czy niepewności, które mogłyby wynikać z powstawania i ewolucji wymagań w trakcie realizacji projektu. W naszym podejściu do tworzenia aplikacji wykorzystaliśmy hybrydową metodykę.

Na samym początku projektu zastosowaliśmy proces prototypowania, co pozwoliło nam swobodnie eksperymentować z różnymi koncepcjami. Ta faza tworzenia wstępnych modeli umożliwiła nam szybkie odrzucenie tych rozwiązań, które nie spełniały naszych oczekiwań. To nie tylko pozwoliło nam zidentyfikować optymalne rozwiązania jeszcze przed głębszym etapem produkcji, ale także skupiło się na stworzeniu wizualnych reprezentacji naszych pomysłów. Wykorzystaliśmy oprogramowanie do tworzenia wireframe'ów, takie jak Balsamiq, co umożliwiło nam stworzenie widoków dla całej aplikacji. Te wizualne prototypy służyły nam do lepszego zrozumienia struktury i funkcjonalności, pomagając w szybkim i skutecznym wyłonieniu najlepszych pomysłów na wczesnym etapie projektowania.

Zamknięcie tego etapu odbyło się poprzez prezentację makiet klientowi i uwzględnienie jego uwag. Dzięki temu procesowi mieliśmy możliwość wdrożenia cennego feedbacku, co pozwoliło nam lepiej dopasować nasze koncepcje do oczekiwań klienta jeszcze przed głębszym etapem produkcji.

W drugim etapie płynnie przeszliśmy do podejścia przyrostowo-iteracyjnego, korzystając z feedbacku z pierwszej fazy, by wprowadzać stopniowe ulepszenia i rozwijać projekt. Ta elastyczność nie tylko pozwoliła nam dopasować się do potrzeb klienta, ale również eksperymentować z różnymi rozwiązaniami, zintegrowanymi w spójny sposób z naszymi wcześniejszymi udoskonaleniami.

Ta hybrydowa metodyka wynikała z naszego pierwotnego podejścia, opartego na jasnych wymaganiach, co umożliwiło nam efektywne budowanie koncepcji systemu. Dzięki temu mieliśmy okazję reagować na nowe potrzeby klienta, unikając potencjalnych błędów i stale podnosząc jakość projektu. To połączenie podejść pozwoliło nam osiągnąć optymalny balans między adaptacją a systematycznym rozwojem, kluczowy dla sukcesu całego przedsięwzięcia.

Dodatkowo początkowe wytyczne klienta dają pewność, że podejmowane kroki są oparte na rzeczywistych potrzebach, a nie jedynie założeniach.

4.2. Osoby i ich role w projekcie

W ramach projektu zaangażowanych było czterech wykonawców, z różnorodnymi obszarami odpowiedzialności. Każda z tych osób aktywnie uczestniczyła w procesie tworzenia kodu i rozwoju aplikacji, przynosząc ze sobą unikalne umiejętności i wkład w projekt.

Dodatkowo, w zespole mieliśmy wyznaczone kluczowe role. Jedną z osób pełniła funkcję Product Ownera - była głosem klienta w zespole. Jej zadaniem było definiowanie i priorytetyzowanie wymagań, wytyczanie kierunku rozwoju produktu oraz zapewnienie, że finalny rezultat spełni oczekiwania klienta.

Z kolei inną osobą miała rolę Team Leadera. Jej zadaniem było zapewnienie, by proces realizacji projektu przebiegał zgodnie z przyjętą metodyką opartą o Scrum. Team Leader dbał o efektywność pracy zespołu, eliminację blokad, a także wspierał i motywował członków zespołu do osiągnięcia wyznaczonych celów.

Wszystkie osoby zaangażowane w projekt aktywnie uczestniczyły w tworzeniu kodu, jednakże w ramach zróżnicowanych odpowiedzialności każda z nich wносиła unikalny wkład w rozwój aplikacji, co przyczyniało się do różnorodności perspektyw i kompetencji w zespole.

4.3. Zespół i podział obowiązków

4.3.1. Forma pracy zespołowej

Burze mózgów były stałym elementem naszego procesu pracy. Przyjęliśmy zasadę, że każda decyzja dotycząca projektu była podejmowana w konsultacji z resztą zespołu. Ta otwarta forma współpracy pozwalała nam czerpać z różnorodnych perspektyw każdego z członków, co sprzyjało generowaniu nowatorskich pomysłów. Wprowadzenie jasnej komunikacji w zespole umożliwiało skuteczne dzielenie się myślami i wspólną pracę nad rozwiązaniami. Dzięki temu każdy członek zespołu czuł się ważnym uczestnikiem procesu decyzyjnego, co zwiększało zaangażowanie i odpowiedzialność za wspólne cele projektowe. Ponadto, systematyczne spotkania sprzyjały budowaniu zespołowej świadomości, co korzystnie wpływało na harmonijne funkcjonowanie.

4.3.2. Sebastian Kozak

Projektowanie bazy danych - w głównej mierze tworzenie bazy danych, opracowanie struktury danych systemu i relacji między danymi.

Programowanie i rozwój funkcjonalności - programowanie backendu, zajmując się logiką biznesową, bezpieczeństwem i efektywnością.

4.3.3. Marcin Mikuła

Projektowanie bazy danych - optymalizacja zapytań i jakości połączenia backendu z bazą danych.

Programowanie i rozwój funkcjonalności - programowanie backendu, zajmując się logiką biznesową, bezpieczeństwem i efektywnością.

Kontakt z klientem - zastępowanie Dominika Grzesika zapewniając stały przepływ informacji między zespołem a klientem.

4.3.4. Szymon Słota

Projektowanie bazy danych - wypełnienie bazy danych danymi testowymi niezbędnymi zarówno podczas tworzenia aplikacji jak i dla wersji aplikacji oddanej do testowania klientowi.

Programowanie i rozwój funkcjonalności - stworzenie aplikacji webowej wraz z jej połączeniem do backendu, zapewniając pełnię funkcjonalności.

Kontakt z klientem - omawianie widoków, aby dostosować oprogramowanie do oczekiwań i potrzeb klienta.

4.3.5. Dominik Grzesik

Bezpośredni kontakt z klientem - zadbanie o przepływ informacji między zespołem programistycznym a klientem oraz zapewnienie zgodności koncepcji obu stron.

Programowanie i rozwój funkcjonalności - rozwijanie aplikacji mobilnej, między innymi programowanie frontendu i jednolicense rozwiązanie dla platform iOS i Android. Pomoc w projektowaniu bazy danych - struktury modeli i relacji.

4.3.6. Zespół

Projektowanie UI aplikacji

W procesie projektowania interfejsu użytkownika (UI), wszyscy członkowie zespołu byli aktywnie zaangażowani. Wspólnie zdefiniowaliśmy i zadbaliliśmy o wygląd i intuicyjność rozwiązania, dbając o spójność między wersją webową a mobilną.

Testowanie i jakość oprogramowania

Wszyscy członkowie zespołu byli zaangażowani w proces testowania, korzystając z wspomnianych wcześniej (3.6) scenariuszy testowych. Testy obejmowały zarówno funkcjonalności backendu, jak i interfejsu użytkownika, zapewniając kompleksowe sprawdzenie jakości oprogramowania.

Kontrola jakości kodu

Proces przeglądania i oceniania jakości kodu miał miejsce w pull requestach w repozytoriach Git'owych. Oprócz tego, istniała otwarta komunikacja między członkami zespołu, umożliwiającą wzajemne oceny i sugestie.

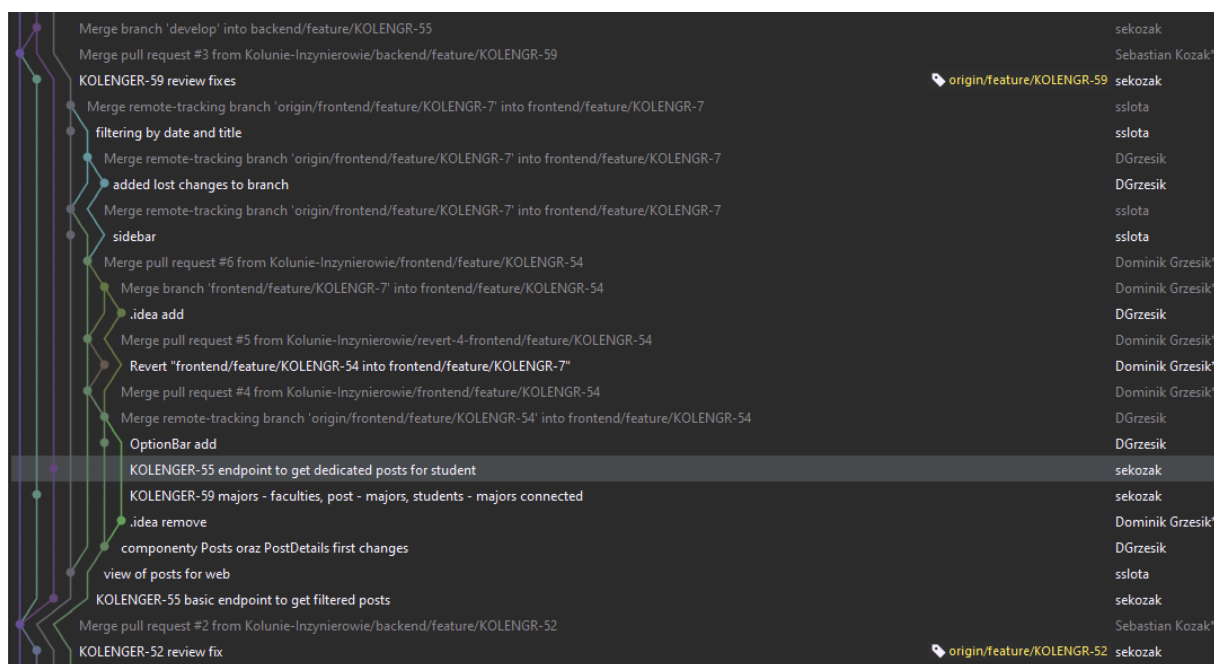
Refactoring

Każdy członek zespołu dbał o kod, co obejmowało zarówno utrzymanie klarowności, jak i konsekwencję w stosowaniu najlepszych praktyk programistycznych. Proces refaktoryzacji był naturalną częścią rozwoju projektu.

4.4. Wykorzystane narzędzia do organizacji pracy

4.4.1. Git

Podstawowym narzędziem z jakiego skorzystaliśmy do tworzenia aplikacji był Git. Git to system kontroli wersji, który umożliwia śledzenie zmian w kodzie źródłowym. Jest to narzędzie do zarządzania kodem, które pozwala programistom rejestrować zmiany, tworzyć gałęzie, łączyć zmiany oraz współpracować z innymi programistami nad projektem. Początkowo korzystaliśmy z niego w połączeniu z Githubem, jednak ostatecznie zdecydowaliśmy się przejść na Bitbucket z racji naszego większego doświadczenia z tą aplikacją i bardziej przejrzystym ui.



Rysunek 4.1: Fragment drzewka roboczego Git z projektu backend

Zamieszczony wyżej zrzut przedstawia fragment drzewa roboczego Git z projektu backend. Projekt rozpoczęliśmy z gałęzi main, na której znajdują się, stabilna, działająca wersja projektu. Z niej wychodziła nowa, osobna gałąź - develop. Develop stanowił wersję rozwojową projektu, do której trafiały wszelkie nowe zmiany. Wszystkie nowe zmiany były implementowane na osobnych gałęziach.

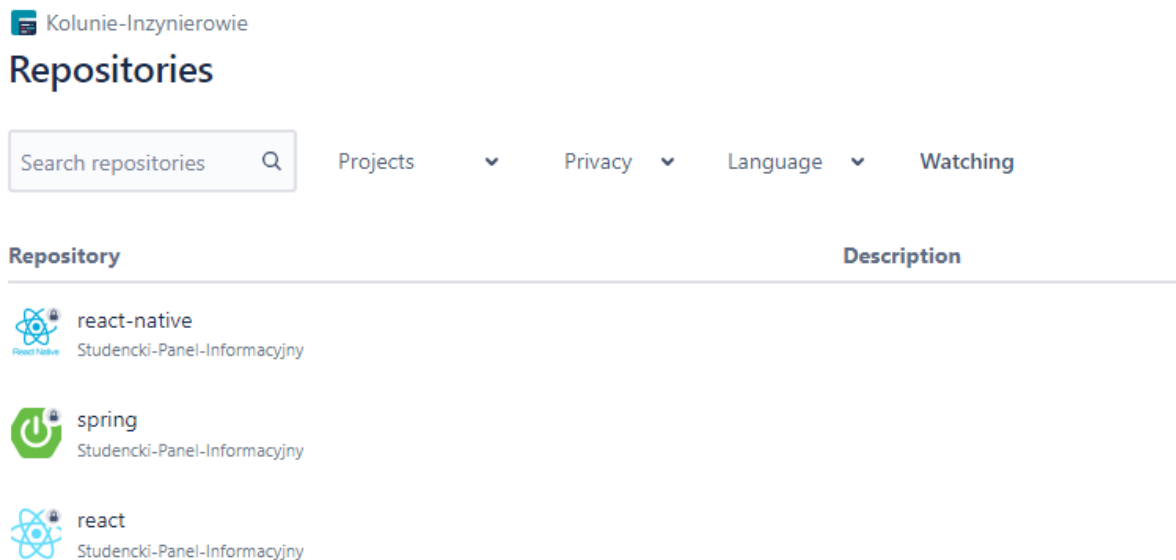


Rysunek 4.2: Przebieg prac nad zadaniami

Przed dołączeniem zmian z gałęzi roboczej do develop, musiały one przejść review co najmniej jednego członka zespołu i dopiero po uwzględnieniu ewentualnych poprawek mogły być dołączone. Załączony zrzut obrazu przedstawia kilka z wielu pull requestów, przy większości z nich są komentarze, niektóre zostały odrzucone z uwagami do poprawy.

4.4.2. Bitbucket, Github

Bitbucket oraz GitHub są platformami internetowymi opartymi na systemie Git, umożliwiające hostowanie kodu źródłowego i projektów, które są zarządzane przy użyciu Git. Są to serwisy internetowe, które działają jako centralne repozytorium dla kodu źródłowego, umożliwiając programistom współpracę nad projektem, przeglądanie kodu, zarządzanie zmianami, tworzenie gałęzi i wiele innych funkcji związanych z systemem kontroli wersji Git.



Rysunek 4.3: Repozytoria na platformie Bitbucket

4.4.3. Jira

W celu płynnego zarządzania zadaniami wykorzystaliśmy Jirę. Narzędzie to umożliwia efektywne śledzenie i zarządzanie zadaniami w projekcie. Dzięki funkcjom takim jak tablice Scrum, backlogi czy raporty, zespół może skutecznie planować, monitorować postępy i dostosowywać strategię.

4.4.4. Teams

Teams umożliwia komunikację i wymianę informacji z klientem. Wirtualne spotkania, czaty i udostępnianie plików pomagają w szybkiej wymianie uwag i pomysłów.

4.4.5. Discord

W naszym zespole najczęściej używaną wersją komunikacji był Discord. Jest to platformę umożliwiającą szybką i łatwą komunikację głosową i tekstową. Wszyscy członkowie zespołu dobrze znają tę platformę, więc większość spotkań tam właśnie się odbywała.

4.4.6. Messenger

W przypadku pilnych pytań, szybkiego uzyskiwania odpowiedzi, wszyscy użytkownicy korzystali z Messengera jak głównego środka komunikacji tekstowej, ze względu na łatwość i wygodę użytku - każdy ma go na własnym telefonie.

4.4.7. Email

Email nadal pełni rolę ważnego narzędzia do formalnej komunikacji w zespole, szczególnie w sprawach dotyczących przesyłania dokumentów i artefaktów.

4.4.8. Google Drive

Google Drive był głównym narzędziem wykorzystywanym do przesyłania ciężkich artefaktów, ze względu na wysoki limit rozmiaru plików. Wykorzystaliśmy go między innymi w celu przekazania klientowi demo aplikacji.

4.4.9. Overleaf, diagrams.net, Balsamic, Dokumenty Google

W momencie projektowania ważną rolę pełniły narzędzia pozwalające na współpracę w czasie rzeczywistym. Overleaf wykorzystany był w celu wspólnego pisania dokumentacji, diagrams.net do wizualizacji struktury systemu, Balsamic to aplikacja, w której powstało UI produktu oraz cała stylizacja, a Dokumenty Google były cennym narzędziem to tworzenia dokumentów pomocniczych, takich jak instrukcja uruchomieniowa demo aplikacji.

4.5. Przebieg prac

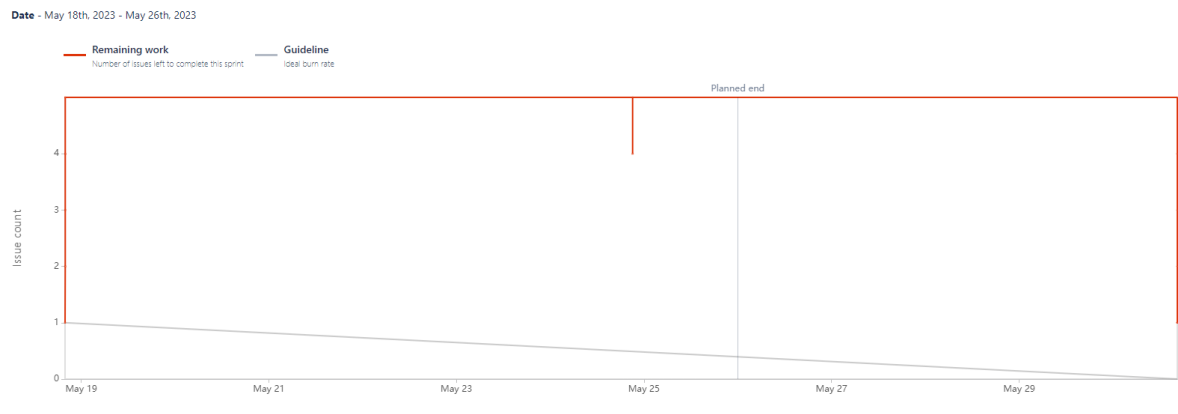
Praca nad projektem podzielona została na 6 widocznych poniżej kamieni milowych:

Kamień Milowy 1 (do 28 kwietnia) - Wizja Produktu

- Badanie dziedziny problemu i ustalenie celów.
- Przegląd istniejących rozwiązań, analiza ich zalet i wad.
- Określenie zakresu funkcjonalności aplikacji.
- Analiza ryzyka.
- Efekt: Ustalenie wizji produktu i jego roli, określenie użytkowników.

Kamień Milowy 2 (do 26 maja) - Koncepcja Produktu

- Opracowanie wymagań funkcjonalnych i нефункциональных dla użytkowników.
- Wybór stosu technologicznego.
- Dokładny opis funkcji użytkownika.
- Stworzenie makiet prezentujących funkcjonalności produktu.
- Efekt: Koncepcja działania produktu, interaktywna wizja produktu, finalna wersja specyfikacji wymagań.

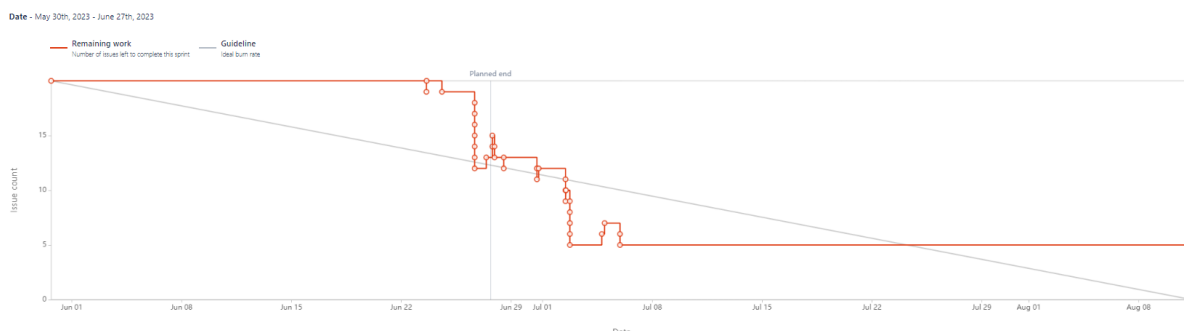


Rysunek 4.4: Burndown Chart dla Sprintu 1

Jak widać na wykresie, istotą kamienia milowego 2 było opisanie funkcjonalności budowanego systemu, utworzenie odpowiadających im stories, a następnie wypełnienie nimi backlog'u w Jirze - tak powstałe zadania były zamykane w następnych sprintach.

Kamień Milowy 3 (do 23 czerwca) - Prototyp Funkcjonalny I

- Implementacja najbardziej podstawowych funkcjonalności aplikacji.
- Efekt: Funkcjonalny prototyp produktu obejmujący podstawowe scenariusze użycia.

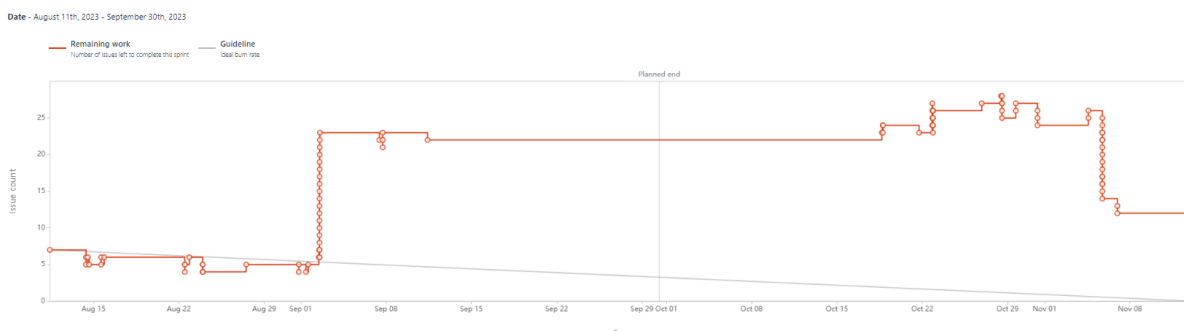


Rysunek 4.5: Burndown Chart dla Sprintu 2

Skoki na wykresie wskazują na to, że mimo przygotowanych w kamieniu milowym 2 zadań, pojawiały się nowe wymagania, zarówno ze strony klienta jak i zespołu, jako skutek analizy stanu projektu i konieczności wprowadzenia zmian. Pozioma linia na końcu sprintu wskazuje na to, że udało się na czas w zaimplementować założone funkcje oraz zakończyć inne zadania, nieobejmujące programowania.

Kamień Milowy 4 (do 6 listopada) - Prototyp Funkcjonalny II

- Implementacja większości funkcjonalności aplikacji.
- Utworzenie funkcjonalnego prototypu produktu obejmującego główne scenariusze użycia.
- Efekt: Funkcjonalny prototyp produktu obejmujący główne scenariusze użycia.

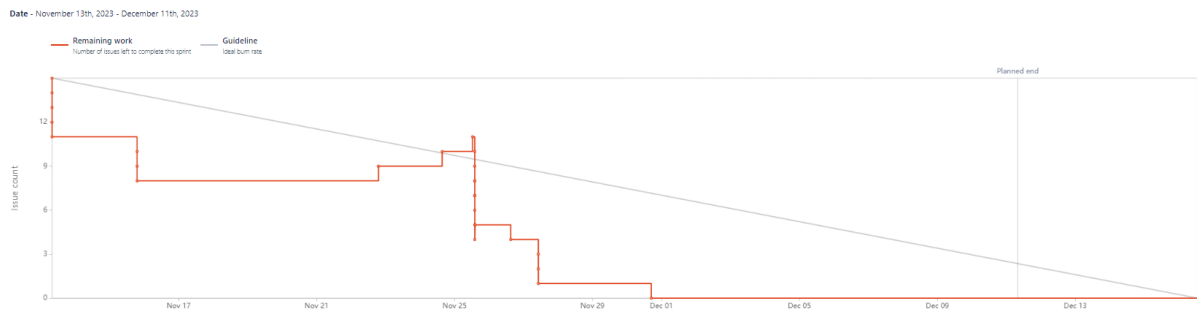


Rysunek 4.6: Burndown Chart dla Sprintu 3

Podobnie jak w kamieniu milowym 3, do backlog'u dodaliśmy znaczną ilość nowych zadań. Wiele z nich obejmowało stylizację aplikacji bądź łączenie backend'u z frontend'ami. Ostatecznie również udało się zakończyć założoną ilość zadań.

Kamień Milowy 5 (do 27 listopada) - Skończony Produkt

- Wykończenie aplikacji - implementacja pozostałych funkcjonalności.
- Udoskonalenie interfejsu użytkownika.
- Stworzenie gotowego produktu i wstępnego draftu pracy inżynierskiej.
- Stworzenie środowiska testowego, które klient może uruchomić lokalnie bez konieczności instalacji narzędzi programistycznych.
- Efekt: Gotowy produkt wraz ze środowiskiem testowym, wstępny draft pracy inżynierskiej.



Rysunek 4.7: Burndown Chart dla Sprintu 4

Na wykresie widać znaczny spadek aktywności, jeśli pojawiały się nowe wymagania, to w niewielkich ilościach, stąd zaledwie 3 skoki na wykresie. Pozioma linia o wartości 0 wskazuje na to, że ostatecznie zdołaliśmy zamknąć wszystkie zadania w wyznaczonym terminie.

Kamień Milowy 6 (do 11 grudnia) - Finalizacja Pracy Inżynierskiej

- Poprawki na podstawie uwag klienta.
- Zamknięcie projektu.
- Efekt: Gotowy produkt - kompletna praca inżynierska, finalna wersja aplikacji i gotowa dokumentacja projektu.

4.6. Występujące ryzyka

Podczas prac nad wizją produktu założyliśmy, że istnieje możliwość wystąpienia wielu rodzajów nieprzewidywalnych zdarzeń mogących mieć wpływ na skuteczność osiągania przez nas wyznaczonych celów.

Nasze założenia obejmowały obszary:

- Wymagania biznesowe:

Ryzyko, że wymagania biznesowe mogą zostać błędnie zinterpretowane, co mogło doprowadzić do nieporozumień między klientem a zespołem.

- Technologia:

Technologia użyta w projekcie (React-Native) była nowa i nieznana dla zespołu projektowego, co mogło doprowadzić do opóźnień i problemów z wdrożeniem projektu.

- Bezpieczeństwo:

Ryzyko braku bezpieczeństwa w systemie związane z brakiem hermetyzacji oraz błędami w implementacji systemu autoryzacji. Brak koniecznych zabezpieczeń może prowadzić do wycieków danych studentów i pracowników uczelni.

- Zespół projektowy:

Ryzyko związane z zespołem projektowym obejmowało niedostateczną wiedzę lub doświadczenie członków zespołu, brak skutecznej komunikacji, konflikty personalne lub niezdolność do realizacji wymagań projektu.

- Terminy:

Terminy realizacji kolejnych kamieni milowych były ważnym elementem projektu aplikacji, a opóźnienia mogły prowadzić do niezadowolenia klienta i konfliktów w zespole.

W ramach prac nad produktem napotkaliśmy dwa ryzyka rozważane podczas tworzenia wizji. Pierwsze związane jest z React-Native, którego zdecydowaliśmy się użyć do stworzenia aplikacji mobilnej. Jako nowa, wcześniej przez nas niepoznana technologia, okazał się on problematyczny przy wdrażaniu wersji testowej dla klienta - na bieżąco uczyliśmy się konfiguracji i obsługi frameworku. Natomiast w kontekście wdrażania aplikacji, ryzyko to nadal istnieje, ponieważ żaden z członków zespołu nie ma doświadczenia ani wystarczającej wiedzy o pracy z technologiami w tym obszarze.

Drugie ryzyko dotyczyło terminów realizacji kamieni milowych, szczególnie pierwszych dwóch. W niewielkim oknie czasowym musieliśmy opracować wizję całej aplikacji, przygotować widoki przedstawiające zaprojektowane przez nas funkcje, przedstawić całość prac klientowi i wprowadzić wymagane przez niego poprawki. Choć udało nam się dotrzymać wymaganych terminów, wysiłek ten wymagał od nas większego nakładu czasu i pracy niż początkowo zakładaliśmy.

Rozdział 5

Wyniki projektu

5.1. Podsumowanie

Podstawowym celem naszego projektu było stworzenie aplikacji usprawniającej przepływ informacji pomiędzy studentami a dziekanatami Akademii Górniczo-Hutniczej w Krakowie.

Ostatecznie udało się zaprojektować i zaimplementować aplikację, która skutecznie rozwiązuje kluczowe problemy komunikacyjne pomiędzy studentami a dziekanatami. Stworzone przez nas rozwiązanie eliminuje rozproszenie informacji po różnych platformach. Dzięki naszemu produktowi, studenci mają dostęp do najważniejszych ogłoszeń związanych z ich wydziałem czy kierunkiem w jednym miejscu, z możliwością filtrowania treści. Dodatkowo aplikacja zapewnia wsparcie dla urządzeń mobilnych, co poprawia dostępność i czytelność komunikatów. Natomiast dla pracowników dziekanatów stanowi efektywne narzędzie przekazywania informacji, co może znacząco wpłynąć na komunikację pomiędzy obiema stronami.

Natomiast nie udało nam się zrealizować wszystkich wymagań niefunkcjonalnych. W naszym projekcie zabrakło testów wydajnościowych, sprawdzających działanie aplikacji w trakcie obciążenia przez dużą liczbę użytkowników.

Co więcej, nie wszystkie założenia projektowe przeszły próby implementacji. Podczas rozwijania projektu, a co za tym idzie podczas zapoznawania się z budową systemu komunikacji akademickiej, doszliśmy do wniosku, że niektóre elementy były zbędne.

Jedną z takich idei były powiadomienia, które otrzymywać miał admin w momencie założenia konta pracowniczego. Administrator miał być odpowiedzialny za zatwierdzanie lub odrzucanie wniosków o utworzenie nowego konta z domeny "agh.edu.pl". Zamiast tego, wymagana jest wyłącznie aktywacja konta przez link wysłany na uczelniany adres mailowy pracownika dziekanatu, wprowadzany podczas rejestracji.

Zmiana koncepcji również dotyczyła aplikacji mobilnej. Początkowo zakładaliśmy, że starosta będzie mógł publikować ogłoszenia tylko dla studentów swojego kierunku i roku, a przewodniczący WRSS dla studentów danego wydziału. Doszliśmy jednak do wniosku, że obaj użytkownicy są przedstawicielami poszczególnych grup, w związku z tym daliśmy im możliwość publikowania komunikatów w całym systemie.

Powstały Studencki Panel Informacyjny skupia najistotniejsze funkcje z punktu widzenia studenta w jednym miejscu, a dla pracowników dziekanatu stanowi efektywny sposób przekazywania informacji. Rozwiązanie jest w stanie poprawić komunikację i usprawnić dostęp do istotnych ogłoszeń i komunikatów, oraz spełniło szczegółowe wymagania klienta.

5.2. Prezentacja wyników projektu

W tym rozdziale przedstawimy wyniki projektu poprzez pokazanie ich możliwości, krok po kroku, na wybranych scenariuszach dla:

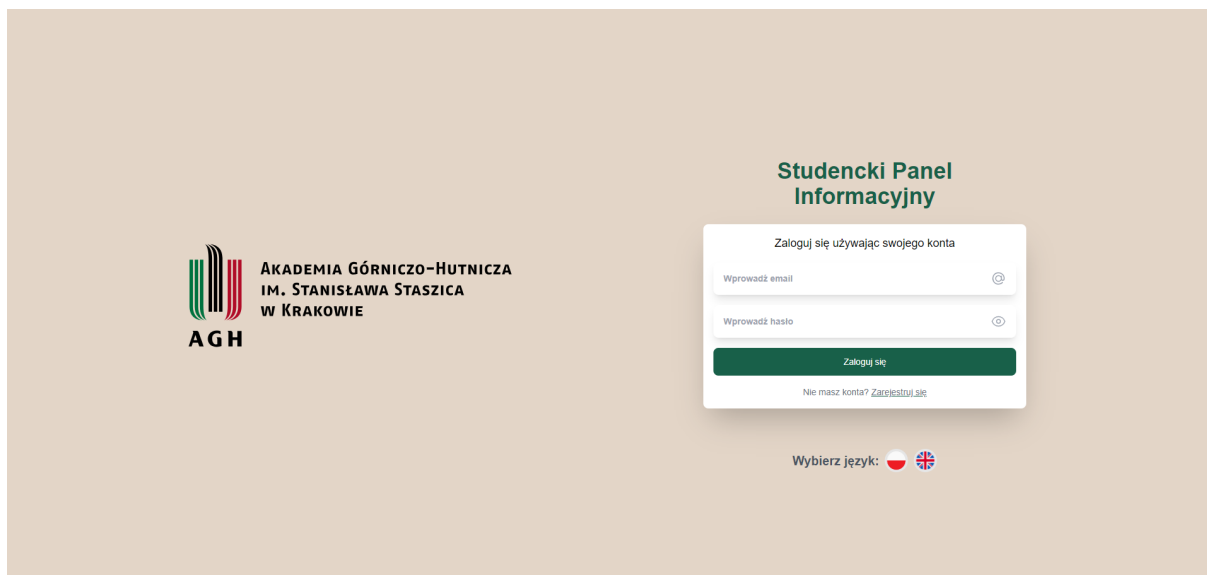
Aplikacji webowej:

- logowanie
- rejestracja
- podgląd na komunikaty i opcję dodania ogłoszenia
- wgląd na szczegóły ogłoszenia
- podgląd profilu i postów opublikowanych przez zalogowanego użytkownika
- edycja profilu
- edycja komunikatu
- widok wylistowanych pracowników
- widok wylistowanych studentów
- widok wylistowanych wydziałów i ich kierunków

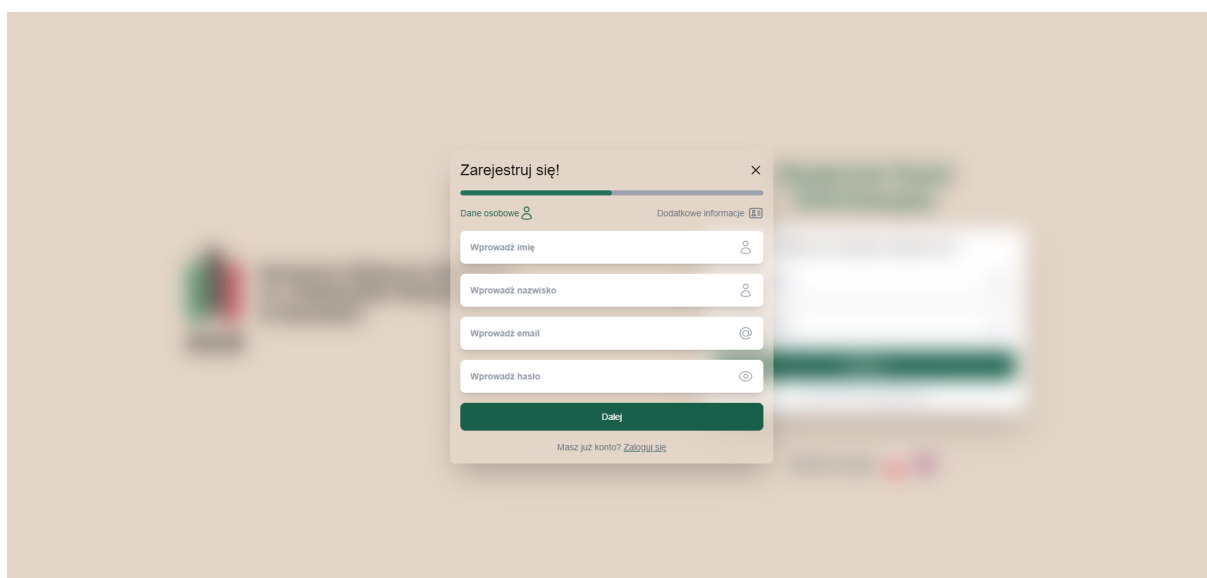
Aplikacji mobilnej:

- rejestracja
- logowanie
- podgląd na komunikaty
- wgląd na szczegóły ogłoszenia
- podgląd na komunikaty na koncie starosty i przewodniczącego WRSS
- dodanie ogłoszenia wraz z wyborem odbiorców
- podgląd na posty opublikowane przez zalogowanego użytkownika
- edycja komunikatu
- podgląd profilu
- edycja profilu
- widok ustawień

5.2.1. Aplikacja webowa

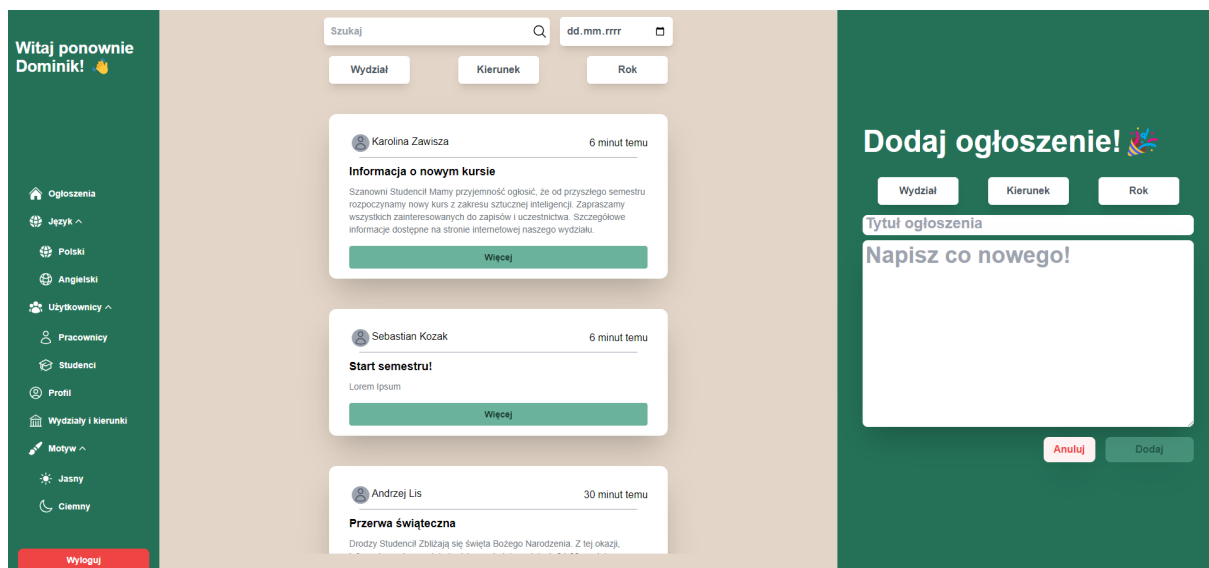


Rysunek 5.1: Logowanie w aplikacji webowej

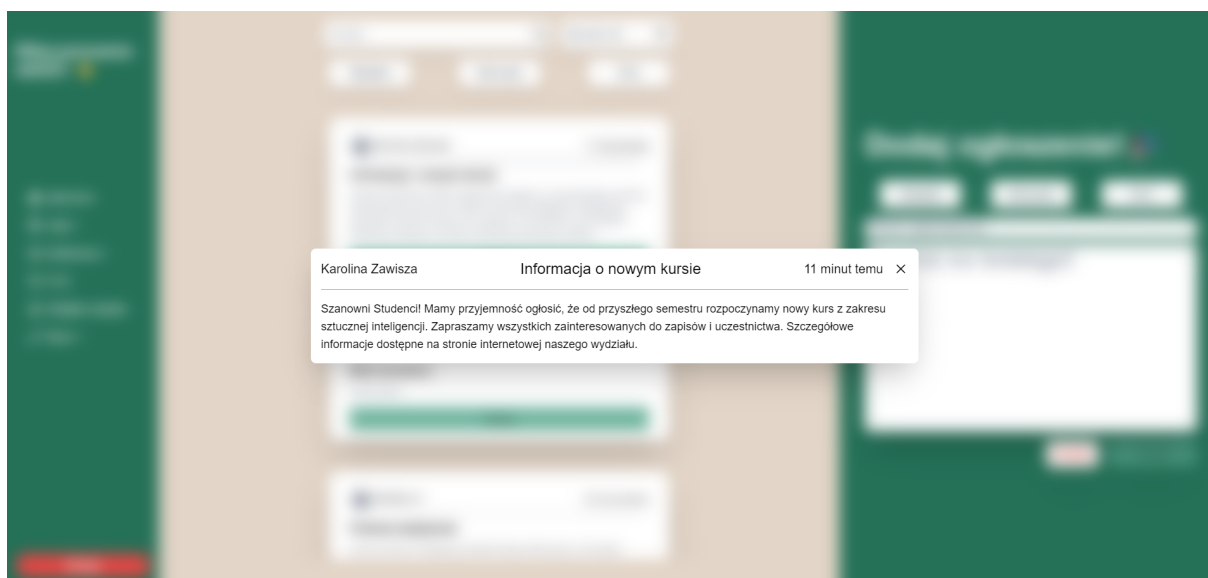


Rysunek 5.2: Rejestracja w aplikacji webowej

Widoczne powyżej zrzuty ekranów przedstawiają formularz rejestracji i logowania dla pracownika w przeglądarce.

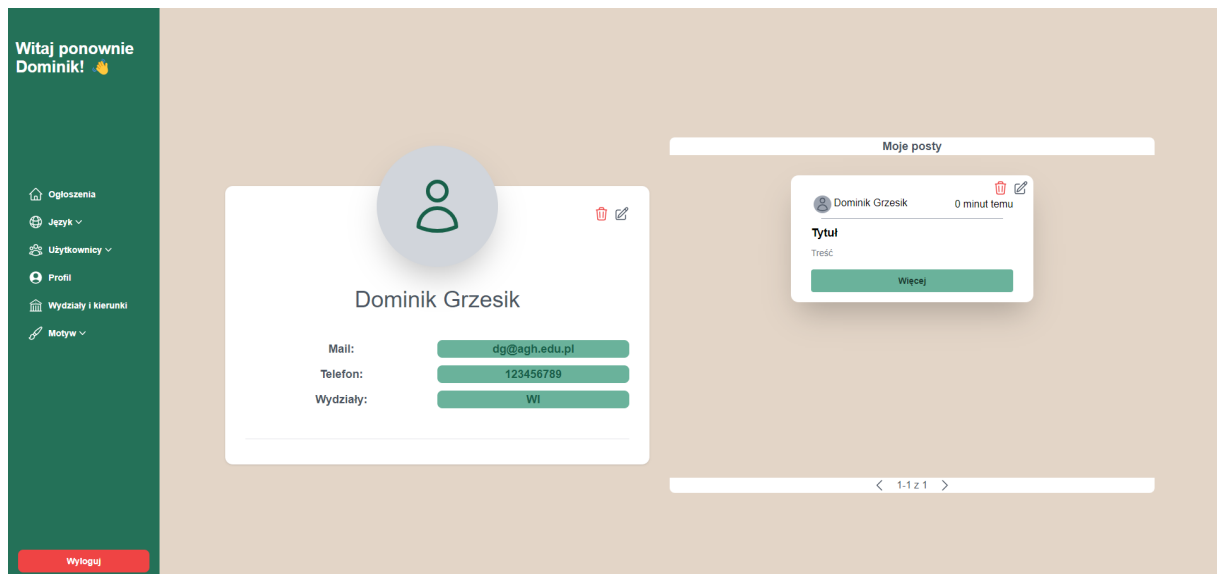


Rysunek 5.3: Podgląd na komunikaty z perspektywy pracownika

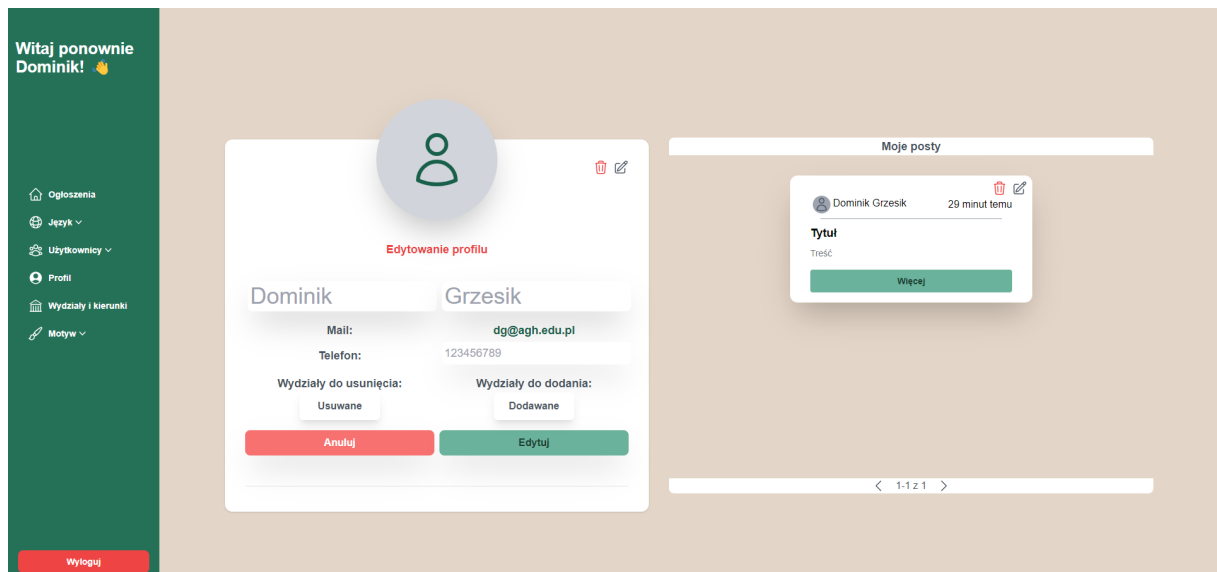


Rysunek 5.4: Podgląd na szczegóły postu

Powyższe zrzuty ekranu przedstawiają główne, najważniejsze widoki aplikacji webowej - posty przeznaczone dla zalogowanego użytkownika z formularzem dodania nowego postu oraz podgląd treści ogłoszenia.



Rysunek 5.5: Podgląd profilu i opublikowanych postów



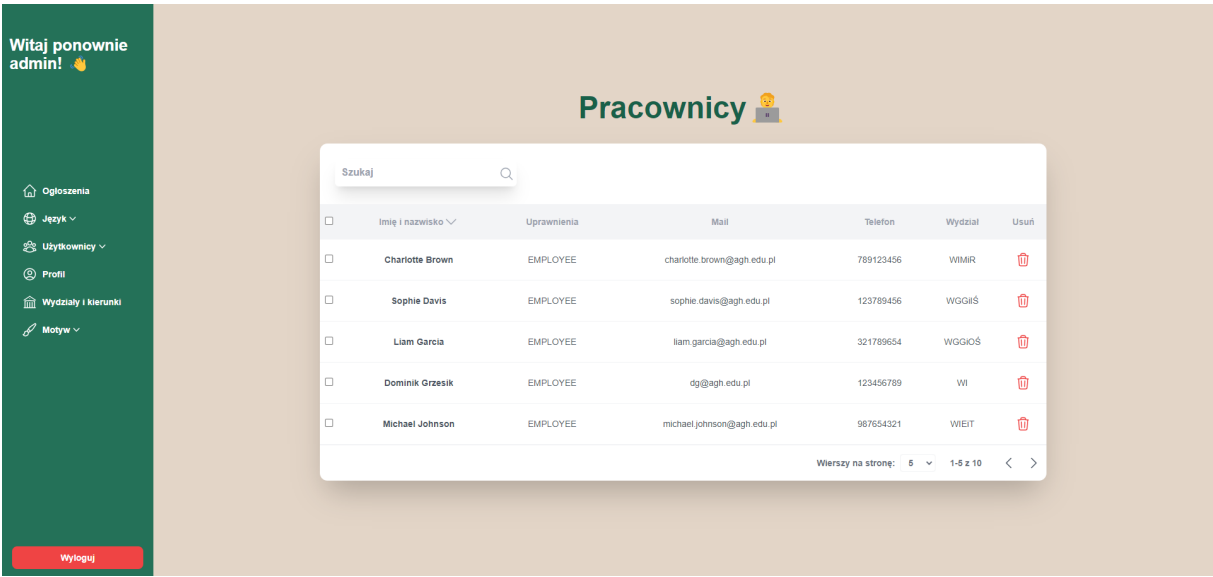
Rysunek 5.6: Edycja profilu

Zrzuty ekranu przedstawiają podgląd profilu użytkownika wraz z listą opublikowanych przez niego ogłoszeń oraz widok edycji widocznych informacji. Z listy można przejść do ekranu edycji komunikatu lub bezpośrednio go usunąć.

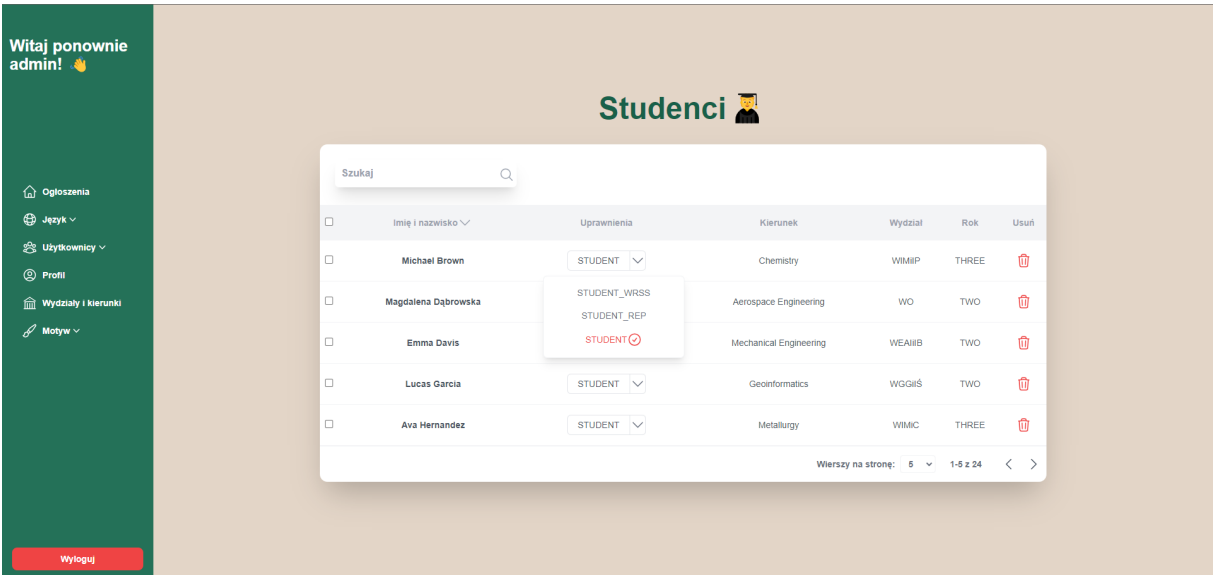


Rysunek 5.7: Edycja komunikatu

Na widocznym wyżej zrzucie przedstawiony jest widok edycji ogłoszenia wybranego z listy komunikatów widocznej na ekranie profilu obecnie zalogowanego użytkownika. Możliwa jest edycja treści i tytułu postu.

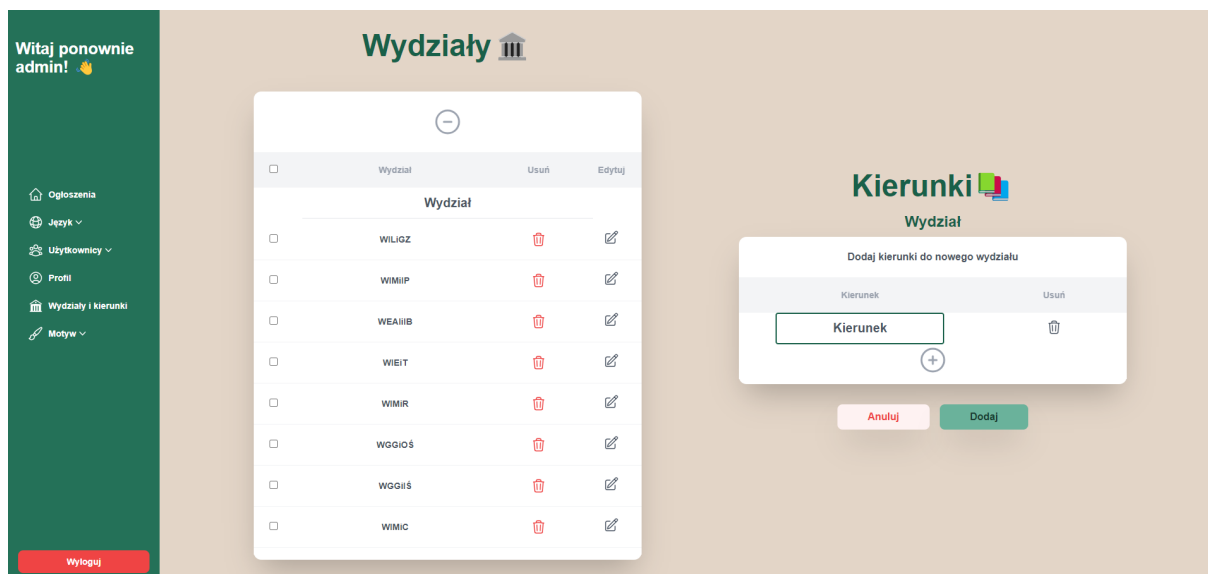


Rysunek 5.8: Widok wylistowanych pracowników



Rysunek 5.9: Widok wylistowanych studentów

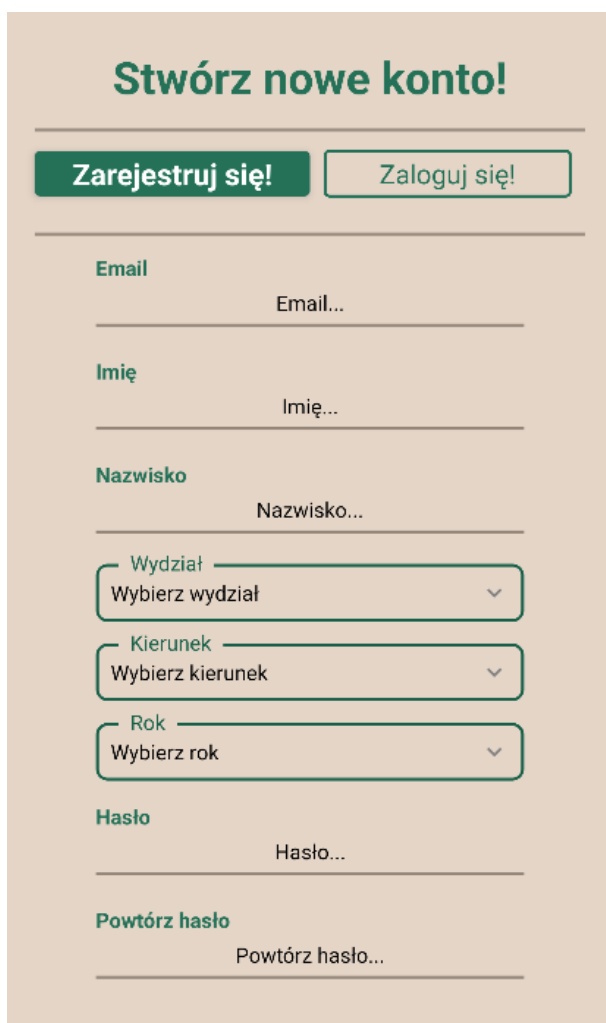
Ekran widoczny powyżej przedstawia widoki listowań odpowiednio pracowników i studentów z perspektywy admina. Admin może zmieniać role studentów i usuwać wszystkich użytkowników. Pracownicy mają wyłącznie wgląd na listy.



Rysunek 5.10: Podgląd na wydziały i kierunki dla wybranego wydziału z opcją edycji

Na widocznym zrzucie ekranu znajduje się widok wydziałów i ich kierunków z perspektywy admina systemu. Może on dodawać, edytować i usuwać kierunki wydziałów. Same wydziały również można dodawać i usuwać oraz zmieniać im nazwy.

5.2.2. Aplikacja mobilna



Stwórz nowe konto!

Zarejestruj się! **Zaloguj się!**

Email
Email...

Imię
Imię...

Nazwisko
Nazwisko...

Wydział
Wybierz wydział

Kierunek
Wybierz kierunek

Rok
Wybierz rok

Hasło
Hasło...

Powtórz hasło
Powtórz hasło...

Rysunek 5.11: Rejestracja na urządzeniu mobilnym




AGH

Witaj!

Email

Hasło

Zaloguj się!

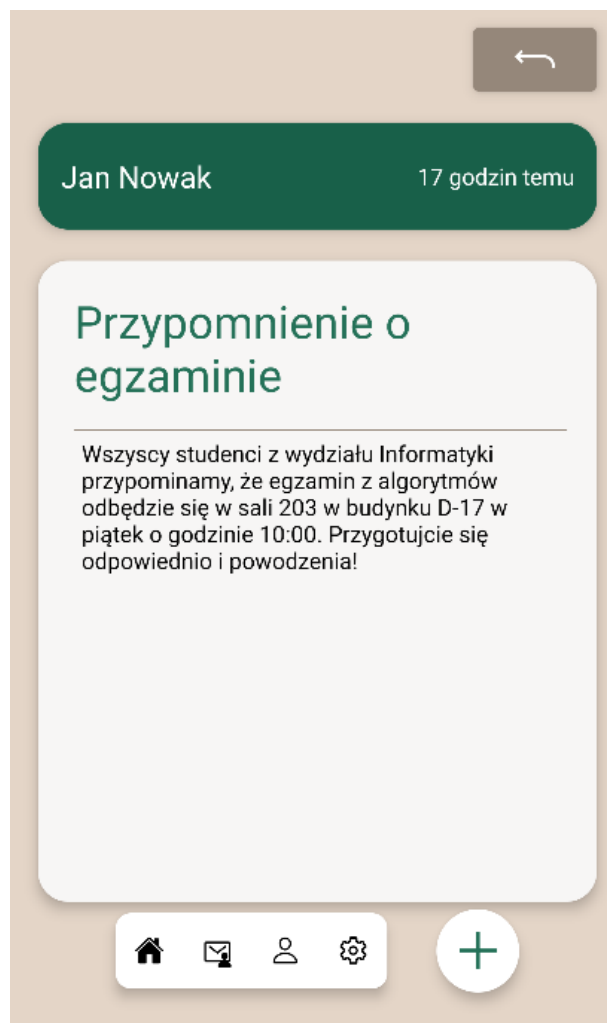
Stwórz nowe konto!

Rysunek 5.12: Logowanie na urządzeniu mobilnym

Widoczne powyżej zrzuty ekranów przedstawiają formularz rejestracji i logowania dla studenta na urządzeniu mobilnym.



Rysunek 5.13: Podgląd postów dla studenta



Rysunek 5.14: Szczegółowy podgląd ogłoszenia

Powyższe zrzuty ekranu przedstawiają główne, najważniejsze widoki aplikacji mobilnej - posty przeznaczone dla zalogowanego użytkownika, oraz szczegółowy podgląd treści ogłoszenia.



Rysunek 5.15: Ogłoszenia z perspektywy starosty i przewodniczącego WRSS

Ten pojedynczy zrzut przedstawia widok postów z perspektywy starosty oraz przewodniczącego wydziałowej rady samorządu studentów. Od poprzedniego różni się tym, że na dole ekranu obok paska akcji widoczna jest ikonka "plus", umożliwiającą dodanie nowego komunikatu.

Dodaj nowy komunikat!

Odbiorcy: 0 >

Tytuł

Napisz o czymś!

Rysunek 5.16: Dodawanie nowego ogłoszenia przez starostę lub przewodniczącego WRSS

Dodaj nowy komunikat!

Wybierz odbiorców: Dodaj

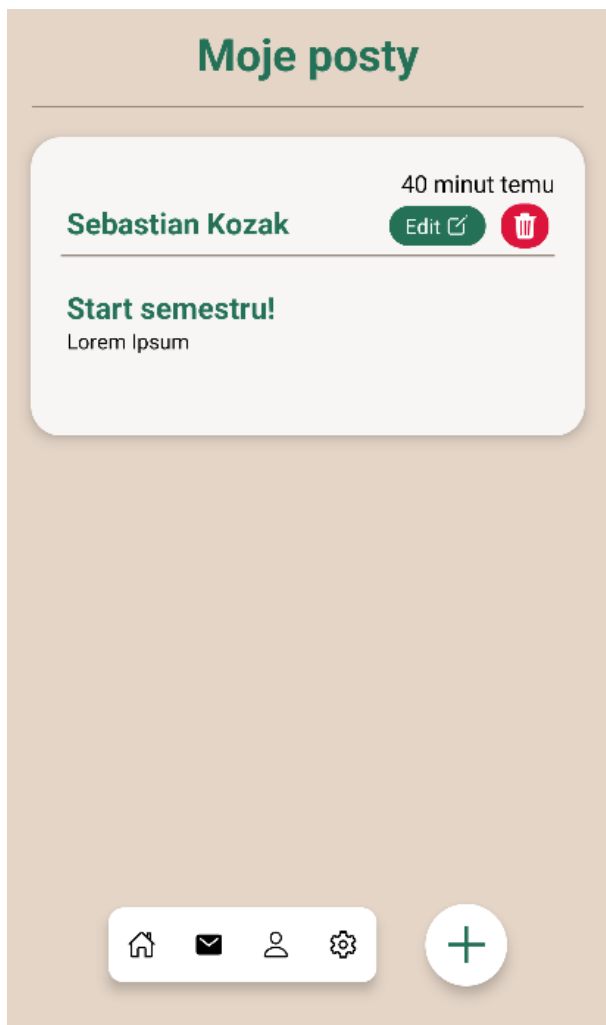
Wydział
Wybierz wydział

Kierunek
Wybierz kierunek

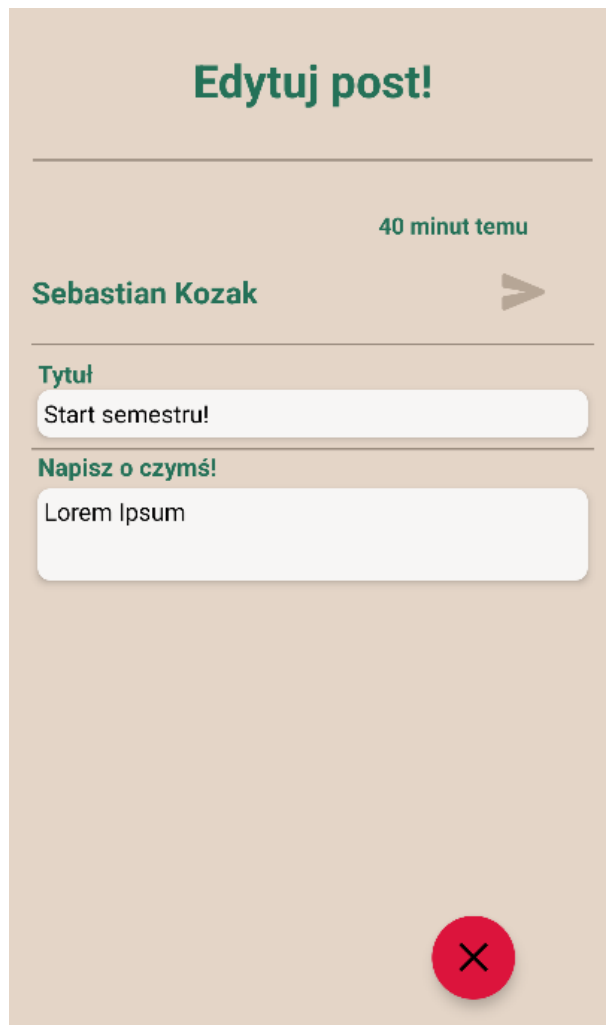
Rok
Wybierz rok

Rysunek 5.17: Wybór grupy docelowej komunikatu

Kolejne, kluczowe widoki dedykowane staroście oraz przewodniczącemu wydziałowej rady samorządu studentów, przedstawiają proces jaki musi przejść użytkownik aby dodać ogłoszenie z urządzenia mobilnego dla wybranej grupy odbiorców.



Rysunek 5.18: Opublikowane posty użytkownika



Rysunek 5.19: Edycja ogłoszenia wybranego z listy

Wybierając kopertę na pasku akcji użytkownik może zobaczyć zamieszczone przez siebie ogłoszenia. Natomiast klikając przycisk "Edit" przy jednym z nich, może on zmienić zawartość komunikatu.

Edytuj

Status

Przewodniczący WRSS

Imię

Sebastian

Nazwisko

Kozak

Email

sk@student.agh.edu.pl

Kierunki:

Computer Science III, WI

Home Messages Profile Settings Add

Rysunek 5.20: Podgląd profilu

Anuluj Zapisz

Imię

Sebastian

Nazwisko

Kozak

Edytuj kierunki:

Dodaj

Wydział

Wybierz wydział

Kierunek

Wybierz kierunek

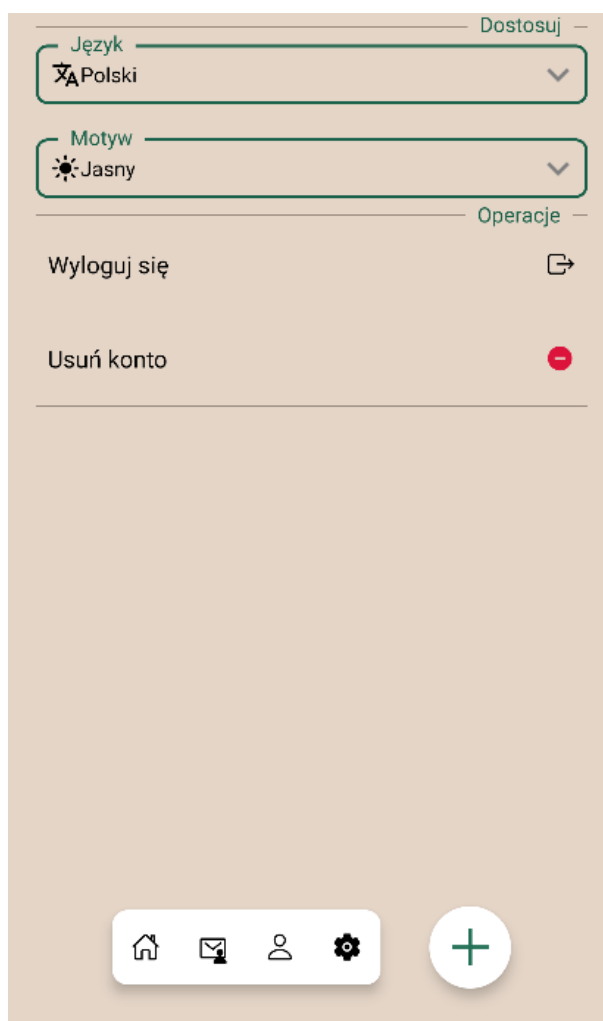
Rok

Wybierz rok

Computer Science III, WI X

Rysunek 5.21: Edycja profilu

Klikając w ikonkę "człowieka" na pasku opcji użytkownik przechodzi do podglądu własnego profilu, widnieje tam jego imię, nazwisko, wydział, kierunek oraz rok studiów. Użytkownik ma możliwość edytowania tych danych, po kliknięciu przycisku Zapisz zostaną one zaktualizowane.



Rysunek 5.22: Widok ustawień aplikacji

Powyższy wklejony przez nas widok przedstawia zakładkę z ustawieniami, można w niej zmienić język wyświetlania w aplikacji, ustawić ciemny lub jasny motyw, wylogować się, a także permanentnie usunąć konto.

5.2.3. Porównanie wyglądu aplikacji na systemach iOS i Android

Stwórz nowe konto!

Zarejestruj się! **Zaloguj się!**

Email
Email...

Imię
Imię...

Nazwisko
Nazwisko...

Wydział
Wybierz wydział

Kierunek
Wybierz kierunek

Rok
Wybierz rok

Hasło
Hasło...

Powtórz hasło
Powtórz hasło...

Rysunek 5.23: Rejestracja w języku polskim na systemie Android

Create your account!

Sign up! **Sign in!**

Email
Email...

First name
First name...

Last name
Last name...

Faculty
Select faculty

Major
Select major

Year
Select year

Password
Password...

Repeat password
Repeat password...

Rysunek 5.24: Rejestracja w języku polskim na systemie iOS

Zrzuty ekranów przedstawiają rejestrację na urządzeniu z systemem iOS w wersji z językiem angielskim, w zestawieniu z ekranem rejestracji w języku polskim na systemie Android. Na pierwszy rzut oka, stylistycznie ekrany mogą wydawać się identyczne, jednakże istnieje między nimi drobna różnica w pogrubieniu czcionki. Choć aplikacja została zaprojektowana tak, żeby działa i wyglądała w taki sam sposób na obu systemach operacyjnych, a użyte w niej style są stworzone przez nas, pojawiła się niewielka różnica w wyglądzie.

5.3. Podsumowanie wraz z własną subiektywną oceną

Podsumowując nasz projekt, jesteśmy bardzo zadowoleni z osiągniętych rezultatów. Cały proces tworzenia przebiegł niezwykle sprawnie, co zawdzięczamy efektywnej organizacji pracy przy użyciu Jiry oraz doskonałej znajomości mocnych stron każdego członka zespołu. Dzięki temu, każdy etap został zakończony zgodnie z harmonogramem, a zamierzone cele były osiągane na każdym etapie postępu projektu.

Ogólny zakres projektu nie uległ znaczącym zmianom od momentu pierwszych ustaleń do finalnej wersji, jedynie niektóre elementy zostały dostosowane, aby uwzględnić nowe pomysły i lepsze rozwiązania. W trakcie tworzenia naszej aplikacji mieliśmy również okazję wymieniać się wiedzą i doświadczeniem, co docenił każdy członek zespołu.

Praca nad produktem od podstaw pozwoliła nam zdobyć doświadczenie i zapoznać się z różnymi technologiami i językami programowania, takimi jak Java, TypeScript, JavaScript, SQL, Spring, Hibernate, Postgres, React, Flutter, React Native, Expo Go, ngrok, Docker, Git oraz Postman. To cenne doświadczenie umożliwiło nam rozwijanie naszych umiejętności i poszerzanie horyzontów w zakresie technologii informatycznych.

Projekt jest elastyczny i otwarty na rozwinięcie o dodatkowe funkcjonalności, dzięki zastosowaniu wzorców projektowych i dbałość o czystość kodu. Istnieje również możliwość dalszego rozwoju poprzez hostowanie aplikacji i podjęcie działań związanych z zapewnieniem bezpieczeństwa w aplikacji.

5.4. Recenzja i opinia klienta

Po przeprowadzeniu zewnętrznych testy użytkowych, otrzymaliśmy opinię od rzeczywistego użytkownika - klienta, dla którego aplikacja została zaprojektowana i zaimplementowana.

Aplikacja spełnia wszystkie początkowo ustalone wymagania oraz realizuje oczekiwany zakres. Wiele zmian zostało naniesionych w trakcie realizacji, po konsultacjach z klientem, zgodnie z założeniami zwinnego zarządzania projektem. Podstawową funkcją aplikacji jest funkcja informacyjna. Panel ma stanowić miejsce do komunikacji między studentami a przedstawicielami dziekanatów, samorządu studenckiego i starostami roku. Identyfikacja wizualna była tworzona w oparciu o wymagania klienta i nawiązuje w stonowany sposób do barw AGH. Najistotniejszymi elementami aplikacji są ogłoszenia kierowane do ściśle określonej grupy docelowej oraz możliwość zarządzania grupami użytkowników aplikacji, nadawania im ról i uprawnień właściwych dla reprezentowanych przez nich funkcji (starosta, przedstawiciel samorządu studenckiego, przedstawiciel dziekanatu) oraz przypisywania studentów do poszczególnych wydziałów, kierunków i lat studiów. Aplikacja działa bez zarzutów zarówno w wersji webowej jak i mobilnej, w różnych przeglądarkach oraz na różnych systemach operacyjnych. Narzędzie zrealizowane na wysokim poziomie, zgodnie z oczekiwaniami klienta, interesariuszy i użytkowników końcowych.

Można więc uznać, że projekt usatysfakcjonował swojego odbiorcę i spełnił swoją rolę, a konieczne konsultacje i zmiany w trakcie jego rozwoju jedynie pozytywnie wpłynęły na jego finalną jakość.

Bibliografia

- [1] *Dokumentacja @react-native-community/datetimepicker*. URL: <https://www.npmjs.com/package/@react-native-community/datetimepicker>.
- [2] *Dokumentacja Eslint*. URL: <https://eslint.org/docs/latest/>.
- [3] *Dokumentacja Expo Secure Store*. URL: <https://docs.expo.dev/versions/latest/sdk/securestore/>.
- [4] *Dokumentacja Expo-CLI*. URL: <https://docs.expo.dev/more/expo-cli/>.
- [5] *Dokumentacja Expo-Go*. URL: <https://expo.dev/client>.
- [6] *Dokumentacja Hibernate*. URL: <https://hibernate.org/orm/documentation>.
- [7] *Dokumentacja JavaMailSender*. URL: <https://docs.spring.io/spring-framework/docs/4.1.6.RELEASE/spring-framework-reference/htmlsingle/#mail>.
- [8] *Dokumentacja JWT*. URL: <https://jwt.io/introduction>.
- [9] *Dokumentacja npm*. URL: <https://www.npmjs.com/>.
- [10] *Dokumentacja PostgreSQL*. URL: <https://www.postgresql.org/docs/>.
- [11] *Dokumentacja React*. URL: <https://react.dev/learn>.
- [12] *Dokumentacja React Native*. URL: <https://reactnative.dev/>.
- [13] *Dokumentacja React Router DOM*. URL: <https://reactrouter.com/en/main>.
- [14] *Dokumentacja react-native-country-flag*. URL: <https://www.npmjs.com/package/react-native-country-flag>.
- [15] *Dokumentacja react-native-element-dropdown*. URL: <https://www.npmjs.com/package/react-native-element-dropdown>.
- [16] *Dokumentacja react-native-vector-icons*. URL: <https://www.npmjs.com/package/react-native-vector-icons>.
- [17] *Dokumentacja sposobów połączeń w React Native*. URL: <https://reactnative.dev/docs/network>.
- [18] *Dokumentacja Spring Security*. URL: <https://docs.spring.io/spring-security/reference>.
- [19] *Dokumentacja Spring-Boot*. URL: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>.
- [20] *Dokumentacja Spring-Data-JPA*. URL: <https://docs.spring.io/spring-data/jpa/docs/current-APSHOT/reference/html/#reference>.
- [21] *Dokumentacja Stack Navigator*. URL: <https://reactnavigation.org/docs/native-stack-navigator/>.

-
- [22] *Dokumentacja Tailwind CSS*. URL: <https://tailwindcss.com/docs/>.
- [23] *Dokumentacja Vite*. URL: <https://vitejs.dev/guide/>.